

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_b7edcaa8-69d\agent-  
a9048172519f599a4.jsonl`  
- SHA-256: `8b51d20ca82169208c6e4014b7a32e7183a04ce6b8ad6559989841747775dea8`  
- Source modified: `2026-06-21T14:42:54+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_b7edcaa8-69d`  
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`
```

Transcript

1. user (2026-06-21T14:39:44.206Z)

Consolidate these implementation tickets into a fresh-context handoff plan. Context: the session already shipped the in-UI bucket?reel CUJ (B-P2) + a screencast; A1 (plain-language copy) is being done now; A2 (WhatsApp/social) is DROPPED (separate scope). Remaining SPA UX = B3/A4/A5/A8 (merge-on-green); then the North-Star runnable-tool (T4?T2?T11) is the CRUCIAL fresh-context work the owner wants paused-for.

TICKETS: [{ "id": "B3", "title": "Tiny shared t(key,vars)/data-i18n shim + language switcher for the site/ marketing surface (en-only), proving one shared catalog across both surfaces", "filesToTouch": ["site/public/i18n.js (NEW ? the ~70-LOC vanilla t()/data-i18n shim + negotiation + switcher wiring)", "site/public/index.html (add <script>, data-i18n attrs, switcher button group, ?lang sync)", "site/public/capture.html (same script + data-i18n attrs + switcher)", "site/public/own-model.html (same script + data-i18n attrs + switcher)", "site/public/locales/en/common.json (NEW ? site catalog, OR a build/copy step that materializes it from the SPA catalog; see changeSpec catalog-home decision)", "site/build- locales.mjs (NEW ? prebuild copy step IF Option A is chosen; see changeSpec)", "site/src/i18n.shim.test.js (NEW ? shim unit tests: t(), interpolation, 2-form plural, data-i18n walk, negotiation order, en-fallback)", "site/src/server.test.js (extend ? Accept-Language: kn / ?lang=kn negotiation parity assertions on the VM host)", "site/src/assets.test.js (extend ? content contract: switcher present, data-i18n attrs present, catalog reachable)", "site/package.json (add a prebuild/test:locales script if Option A; add test:i18n to the suite)"], "changeSpec": "GOAL (tracker \$7 B3): carry the SPA's trilingual win to the vanilla site/ marketing surface via a tiny shared t() shim + data-i18n plumbing, en-only first (kn/hi catalogs already exist for the SPA and ride this once wired). Proves the SHARED catalog works across both surfaces and locks the ONE negotiation contract. Branch from origin/master; ONE PR; no stacking.\n\n=== A. THE t()/data-i18n SHIM (~70 LOC, lives in site/public/i18n.js, plain ES module, zero deps) ===\n\nThe site is DEPENDENCY-FREE and has NO build step: wrangler [assets] serves ./public raw (wrangler.toml:13-15) and server.js reads files off disk (server.js:49-67). So the shim must be a static <script type='module' src='./i18n.js'> with no bundling. Design:\n\n1. CATALOG LOAD: `fetch('/locales/'+lng+'/common.json')` (flat namespace = the SPA's enCommon shape, web/src/i18n/locales/en/common.json) with an en fallback fetch; cache in-module. Keys are dotted paths (e.g. 'app.title', 'footer.count') resolved by walking the nested JSON ? MIRRORS the SPA's i18next default-namespace lookup.\n\n2. t(key, vars): walk dotted key ? string; if missing in active locale, look it up in the en catalog (NEVER return the raw key ? the by-design invariant, FRIENDLY \$4 / I18N_PLAN \$2.3). Then:\n\n- INTERPOLATION: replace `{name}` tokens from vars, matching i18next's `{var}` syntax used across the SPA catalog (mode.loading '{videoId}', query.notAvailableTitle '{token}', result.download '{filename}').\n\n- ICU 2-FORM PLURALS: support the `{count, plural, one {# rally} other {# rallies}}` form already in the SPA catalog (footer.count, selection.count). en/kn/hi are ALL CLDR 2-form (one/other) ? see FRIENDLY \$3 ? so the shim only needs an Intl.PluralRules(lng).select(count) ? pick the `one`/`other` arm ? substitute `#`?count. (Do NOT hand-roll count>1?'s':' ? that's exactly what I18N_PLAN \$2.1 says ICU replaces.) Keep it to the 2-form subset; richer ICU is a documented non-goal (the catalog only uses plural, never select/ordinal).\n\n- HTML-bearing values: the SPA catalog stores inline markup (e.g. app.intro has /<i>, mode.* have <code>). For data-i18n on a host element

use innerHTML for those; provide a data-i18n-text variant (textContent) for attribute/plain contexts. Sanitization is unnecessary (catalog is first-party, build-time content), but DOCUMENT that the catalog is trusted-source-only.

3. data-i18n WALK: on load + on languageChange, querySelectorAll('[data-i18n]') ? el.innerHTML = t(el.dataset.i18n, parseVars(el)); querySelectorAll('[data-i18n-attr]') for attributes like aria-label/placeholder (format: `data-i18n-attr="aria-label:lang.label\`"). vars come from data-i18n-vars (JSON) or data-* ? for the static marketing pages most strings are var-free, so this is mostly a flat text swap.

4. NEGOTIATION (one order, IDENTICAL to the SPA ? web/src/i18n/index.ts:48-53 detection.order ['localStorage', 'querystring', 'navigator'] + fallbackLng 'en'): persisted pref (localStorage key 'khelsutra.lang' ? the SAME LANG_STORAGE_KEY exported by the SPA, web/src/i18n/index.ts:24, so a choice carries across surfaces) ? ?lang=<code> ? navigator.languages/navigator.language ? 'en'. Normalize region subtags (kn-IN?kn) by splitting on '-' (mirrors SPA's nonExplicitSupportedLngs + the LanguageSwitcher base-subtag compare). SUPPORTED = ['en', 'kn', 'hi']; anything else ? 'en'. On switch: changeLanguage(lng) sets localStorage, re-walks data-i18n, sets document.documentElement.lang = lng (a11y/SEO ? mirrors SPA syncDocumentLang, index.ts:59-65), and updates ?lang in the URL via history.replaceState so a shared/reloaded link is sticky.

5. SWITCHER: a small button group (en / ????? / ?????? autonoms from lang.en/kn/hi keys) added to each page header/footer, icon-free, non-color active state via aria-pressed ? copied 1:1 from the SPA's LanguageSwitcher.tsx contract (aria-pressed, lang attr, base-subtag active compare). Place in the index.html header nav (alongside the existing .nav-links, index.html:231-238) and the footer of capture/own-model.

=== B. CATALOG-HOME DECISION (RECOMMENDED: Option A ? hoist to a shared top-level locales/ and COPY into site/public at build) ===

CONTEXT: catalogs live ONLY at web/src/i18n/locales/{en,kn,hi}/common.json today, imported via RELATIVE paths in web/src/i18n/index.ts:13-15. There is NO top-level locales/ yet (the FRIENDLY §4 / I18N_PLAN §2.2 diagrams SHOW a shared top-level locales/, but it doesn't exist ? the SPA owns the only copy). D9/D-CATALOG-HOME (FRIENDLY §2) says 'co-locate locales/ in khelsutra, consumed by BOTH'. The site needs the JSON at a URL it can fetch(); the SPA needs it as an ESM/JSON import bundled by Vite.

Three options weighed:

- Option A (RECOMMEND): HOIST to repo-root /locales/{en,kn,hi}/common.json as the single source of truth. SPA points its 3 imports at ../../../../locales/... (or a Vite alias '@locales'). Site gets a TINY prebuild (site/build-locals.mjs: cp -r ../locales site/public/locales, run via an npm 'prebuild'/'predeploy' + a test:locales freshness check) so wrangler [assets] and server.js serve the JSON at /locales/<lng>/common.json with NO runtime cross-dir coupling. Pros: ONE canonical catalog (honors D9 literally), CI key-parity (B2) runs once over the shared dir, the site stays dependency-free at RUNTIME (the copy is a build/deploy step, not a server dep). Cons: introduces a small site build step + a 'don't edit the copy' guard test (assert site/public/locales/en/common.json byte-equals /locales/en/common.json so a stale copy fails CI ? mirrors the deploy-config.test.js drift-guard philosophy, deploy-config.test.js:1-5).
- Option B (REJECT): site fetches/imports FROM web/src/i18n/locales directly. Breaks because web/src is NOT under site/public (wrangler serves only ./public; server.js PUBLIC-jails paths, server.js:57-58) so the browser can't fetch it, and it couples the marketing deploy to the SPA's source tree.
- Option C (fallback if owner wants zero site build): keep catalogs in web/src/i18n/locales as source of truth and copy to site/public/locales in the same prebuild ? same mechanics as A minus the hoist. Simpler diff (no SPA import re-point) but leaves the 'home' inside the SPA, contradicting the D9 'shared top-level' intent and the §4 diagram. Choose A unless the owner prefers minimal churn.

BUILD/BUNDLING IMPLICATION for site/: the ONLY change to the site's runtime story is that /locales/** must be present in public/ at serve time. With Option A/C, add an npm 'prebuild' (and document it in the deploy path ? Cloudflare Workers Builds runs `npm run build`-style hooks; the VM host serves whatever's in public/). The shim's runtime fetch('/locales/...') then works on BOTH hosts with no server.js change beyond the existing static handler (it already serves .json via TYPES ? NOTE: server.js TYPES map (server.js:26-30) has NO '.json' entry, so JSON falls to 'application/octet-stream'; ADD '.json': 'application/json' to TYPES so fetch() parses cleanly and the Accept-Language test can assert the content-type, mirroring the video/mp4 content-type assertion in server.test.js:62).

=== C. NEGOTIATION PARITY (must match the SPA exactly) ===

Persisted pref (localStorage 'khelsutra.lang') ? ?lang= ? Accept-Language/navigator ? en, base-subtag normalized, en-fallback for unsupported AND for any missing key. This is the SAME order proven for the SPA in web/src/i18n/i18n.test.tsx (N1/N3-N7) and the I18N_PLAN §2.3 contract. The shim's browser-side signal is navigator.languages; the SERVER-side signal (for SSR-less static pages) is the Accept-Language header ? see test plan N8.

newI18nKeys: [], "testPlan": "Mirror the existing node:test conventions (site/src/*.test.js, run via `node --test`; jsdom is already a devDependency used by ux.dom.test.js).

n1. SHIM UNIT (NEW site/src/i18n.shim.test.js, jsdom like ux.dom.test.js:11): import the shim as a module (or boot a JSDOM with a page that loads /i18n.js + a stubbed fetch returning the real en/kn catalogs). Assert:

- t('app.title') returns the en string, NOT the raw key.
- INTERPOLATION:

```

t('result.download',{filename:'reel.mp4'}) === 'Download reel.mp4';
t('mode.loading',{videoId:'abc'}) contains 'abc'.\n - 2-FORM PLURAL:
t('selection.count',{count:1}) === '1 rally selected'; {count:3} === '3 rallies selected' ?
EXACTLY mirroring the SPA assertion in i18n.test.tsx:77-78. Same for footer.count one/other.\n
- data-i18n WALK: build a fragment with <h1 data-i18n="\`app.title\`"></h1> + <button
data-i18n-attr="\`aria-label:lang.label\`">, run the walk, assert innerHTML/aria-label
populated.\n - en-FALLBACK: with active=kn and a key absent from kn (or unsupported
active='ta'), t() returns the en value ? never a raw key (FRIENDLY §6.5 N7).\n\n2. NEGOTIATION
ORDER UNIT (same file): stub localStorage/navigator/?lang and assert resolution: localStorage
'kn' beats navigator 'en' (N4); ?lang=kn beats navigator (N3); navigator 'kn-IN' ? 'kn' base
(N1/N5); navigator 'ta-IN' ? 'en' (N6). These are the static-shim twins of
i18n.test.tsx:85-128.\n\n3. ACCEPT-LANGUAGE SERVER TEST (extend site/src/server.test.js ? boots
the REAL server.js over HTTP, server.test.js:30-41): the §6.5 N8 row. Decide the server contract
FIRST:\n - If pages stay client-negotiated (recommended, zero server templating): assert `GET
/ with header 'Accept-Language: kn'` returns 200 + the page + the shim+catalog are REACHABLE
(fetch /locales/kn/common.json ? 200, content-type 'application/json'), so the client shim can
honor kn. Assert ?lang=kn likewise serves the page and the kn catalog is fetchable. This proves
the SAME negotiation INPUTS the SPA uses are available on the site host.\n - Document that
true server-side language SELECTION (returning a kn-rendered HTML body from Accept-Language) is
OUT OF SCOPE for the static shim (it's client-side); N8's bar is 'the kn page renders given
Accept-Language: kn' which the client shim satisfies once the catalog is served. If the owner
wants server-rendered negotiation, that's a server.js change (read req.headers['accept-
language'] ? inject a <html lang> + a data-initial-lang attribute) ? note it as a small follow-
up, NOT this PR.\n - Also add the '.json' content-type assertion (parity with
server.test.js:62's video/mp4 check) and a /locales freshness/presence check on the VM
host.\n\n4. CONTENT CONTRACT (extend site/src/assets.test.js, the static-integrity guard,
assets.test.js:1-4): assert each of index/capture/own-model.html (a) loads /i18n.js, (b) carries
data-i18n attributes (the switcher will be dead otherwise), (c) the switcher button group is
present with the three autonyms + aria-pressed, and (d) site/public/locales/en/common.json
exists and is non-empty (and, for Option A, byte-equals the root /locales/en/common.json ? the
drift guard).\n\n5. CI: add test:i18n to the suite; ensure the B2 key-parity check covers the
shared catalog home (whichever option). Run the FULL site suite (`node --test site/src/`) AND
the web suite green; confirm no regression in the existing assets/server/ux tests.\n\nBROWSER-
VERIFY (owner machine, since fonts/Indic rendering can't be eyeballed in CI per FRIENDLY §6.5):
load /, /capture, /own-model; switch en?kn?hi; confirm copy swaps, <html lang> updates, the
choice persists across a reload AND carries into the SPA (shared localStorage key); set the
BROWSER language to kn and load the site cold ? kn auto-renders (N1 end-to-end). kn/hi Noto
rendering rides B4's font bundle ? flag if the site needs the same @fontsource Noto self-host
the SPA got (B4 was SPA-scoped; the site may need its own font <link>/self-host ? call this out
as a dependency).","gotchas":["The site is RUNTIME dependency-free with NO bundler ? the shim
MUST be a plain static ES module under public/, loaded via <script type=module>. Do NOT
introduce i18next/a bundler on the site (that's the SPA's stack); the whole point of B3 is the
~70-LOC vanilla twin.","server.js TYPES map (server.js:26-30) has NO '.json' entry ? JSON
currently serves as application/octet-stream. Add '.json':'application/json' or fetch().json()
and the content-type assertion will behave oddly across hosts.","Catalogs live ONLY in
web/src/i18n/locales today (NOT a top-level locales/, despite the §4/§2.2 diagrams). web/src is
OUTSIDE site/public and server.js PUBLIC-jails paths (server.js:57-58), so the site physically
CANNOT serve the SPA's copy ? a copy/hoist step is mandatory, not optional.","Use the SAME
localStorage key as the SPA: 'kheIsutra.lang' (LANG_STORAGE_KEY, web/src/i18n/index.ts:24) ? a
different key silently breaks the cross-surface 'choice carries over' promise (D9 intent).","The
SPA catalog stores HTML inside values (app.intro <b></b>, mode.* <code>, query.support <x>).
Plain textContent swaps will show literal tags; use innerHTML for those keys (catalog is trusted
first-party content) and a textContent variant for attributes ? don't blanket-textContent
everything.","Two CLDR-2-form languages only (en/kn/hi all one/other per FRIENDLY §3) ?
implement ONLY the {count,plural,one}{other{}} subset via Intl.PluralRules; a full ICU parser is
over-engineering and a documented non-goal (the catalog never uses
select/ordinal/nesting).","BOTH hosts must serve /locales/**: Cloudflare [assets]
(wrangler.toml) serves public/ statically AND the VM server.js static handler must too ? verify
the prebuild copy lands in public/ before deploy on the git-connected Workers Builds path
(deploy-config.test.js guards this kind of deploy drift).","Indic FONT for the site: B4 self-
hosted Noto for the SPA only. The marketing pages currently load just Plus Jakarta Sans
(index.html:23). kn/hi will tofu on devices without a system Indic font until the site gets its
own Noto self-host/<link> ? surface this as a dependency (likely a tiny A3/B4-followup, not
blocking the en-only B3 scaffold).","Don't let the switcher be the only thing localized:
assets.test.js must assert real data-i18n attrs exist on content, or a future edit that drops

```

them passes CI while shipping an English page with a dead switcher.]", "effort": "medium", "gated": "No (actionable)", {"id": "A4", "title": "Mobile-first touch targets + tooltip text made visible on phones (SPA)", "filesToTouch": ["web/src/components/QueryBar.tsx", "web/src/components/RallyGrid.tsx", "web/src/components/SelectionFooter.tsx", "web/src/components/LanguageSwitcher.tsx", "web/src/components/IngestPanel.tsx", "web/src/App.tsx", "web/src/i18n/locales/en/common.json", "web/src/i18n/locales/kn/common.json", "web/src/i18n/locales/hi/common.json", "web/src/components/RallyGrid.test.tsx (new)"], "changeSpec": "GOAL: every interactive control meets the ~44px touch-target floor on phones, and the two native title= tooltips (invisible on touch ? no hover) become visible inline text. Doc lever: FRIENDLY_AND_MULTILINGUAL.md §4 row 'Mobile-first touch targets; tooltip-text -> visible' (high impact / medium effort) and §3 HCI4D 'large touch targets, mobile-first'. Scope per §6: usable-on-phone, NOT pixel-perfect.\n\nTOUCH TARGETS ? add `min-h-11` (44px) plus a left-aligned flex centering where needed, to these exact elements (current padding noted):\n- QueryBar.tsx:44-51 text input (`px-4 py-2`) -> add `min-h-11`. QueryBar.tsx:52-58 Apply button (`px-4 py-2`) -> add `min-h-11`. QueryBar.tsx:85-94 example chips (currently `px-3 py-1 text-xs`, the smallest tap targets in the app) -> change to `px-3 py-2 text-sm min-h-11 inline-flex items-center`. QueryBar.tsx:84 `query.try` label is non-interactive ? leave it.\n- App.tsx:247-249 the three bulk buttons (`px-3 py-1`) -> add `py-2 min-h-11 inline-flex items-center` (drop the `py-1`).\n- RallyGrid.tsx:33-44 the card button is already tall (p-3 + multi-line) ? leave its size; it is the primary tap target. Do NOT shrink it.\n- SelectionFooter.tsx:60-67 reel-name input (`px-3 py-1.5`) -> add `min-h-11`. SelectionFooter.tsx:70-77 Build button (`px-5 py-2`) -> add `min-h-11`. LanguageSwitcher.tsx:36-43 locale buttons (`px-3 py-1.5`) -> add `min-h-11 inline-flex items-center`. (Keep icon-free + aria-pressed per its existing contract.)\n- IngestPanel.tsx:43-51 input (`px-4 py-2`) + :52-58 submit + :60-67 cancel buttons -> add `min-h-11` to each.\n\nTOOLTIP -> VISIBLE TEXT (two native title= attrs, unreachable on touch):\n1. QueryBar.tsx:68-76 unavailable chip uses `title={t('query.notAvailableTitle', ...)}`. KEEP the title (desktop hover + a11y) but the chip already shows visible `{token} · not available yet` text, so the meaning is on-screen. No new copy needed here ? the title is supplementary, not the only carrier. Verify and leave as-is, OR (preferred for parity with #2) drop reliance on hover by keeping the existing visible chip text unchanged.\n2. RallyGrid.tsx:60-69 the shots chip carries its caveat ONLY in `title={t('grid.shotsTitle', ...)}` ? invisible on phones. FIX: render the experimental caveat as visible helper text. The chip text already says `~{count} shots · experimental` (grid.shots), which carries the headline caveat visibly; keep the title for the longer 'may be inaccurate / filter by length instead' detail on desktop. This already satisfies 'never tooltip-only' because the word 'experimental' is on-screen. The load-bearing change is therefore the QueryBar example chips + the global min-h-11 sweep; document that both title= attrs were audited and each has a visible counterpart (so no information is hover-gated).\n\nNOTE: no NEW user-facing strings are strictly required if the audit confirms both tooltips already have visible counterparts (they do). Only add keys if the team decides to surface the full grid.shotsTitle text inline (optional `grid.shotsHelp`). Recommend NOT adding copy ? keep this a pure CSS/markup PR to minimize catalog churn and avoid serializing with A5/A8 on the JSON.", "newI18nKeys": [], "testPlan": "1. Extend i18n/hygiene.test.ts no-bare-strings list already covers these files ? no change. 2. Add a focused render test (RallyGrid.test.tsx, @vitest-environment jsdom, mirror IngestPanel.test.tsx setup: import i18n, changeLanguage('en'), render with @testing-library/react) asserting: the rally card button is present, the shots chip text contains 'experimental' (proving the caveat is visible, not hover-only). 3. Optional QueryBar render test asserting each example chip button has class 'min-h-11' (or use getByRole('button') and check className) ? lightweight regression pin for the tap-target fix. 4. Manual: load the SPA in a ~380px viewport (browser devtools), confirm chips/buttons are comfortably tappable and the shots caveat reads without hover. 5. Run `npm run test` + `npm run typecheck` + `npm run build` green.", "gotchas": ["Tailwind v4 (index.css uses @import 'tailwindcss') generates `min-h-11` = 2.75rem = 44px out of the box; no config needed. Verify `min-h-11` actually emits (v4 arbitrary-free spacing scale includes 11).", "Do NOT convert the RallyGrid native title= to a hover-only popover ? the whole point is touch users get the info; keep the visible 'experimental' text as the primary carrier.", "no-bare-strings hygiene test (BARE_ATTR regex) only checks placeholder/aria-label/title ? adding class-only changes won't trip it, but if you add any NEW title=aria-label it MUST go through t().", "This PR touches App.tsx (bulk-button classNames) and the catalogs are touched ONLY if optional keys are added ? to keep it independent of A5/A8, ship A4 with ZERO catalog edits (recommended). If copy is added it MUST serialize behind/ahead of A5/A8 on en/kn/hi.", "grid-cols min card width is 150px (RallyGrid.tsx:25) ? fine on phones; don't shrink it chasing density."], "effort": "medium", "gated": "No (actionable)", {"id": "A5", "title": "Friendly, localized engine-error mapping for the Build/compile path (reuse classifyError)", "filesToTouch": ["web/src/App.tsx", "web/src/components/IngestPanel.tsx", "web/src/lib/errors.ts", "web/src/i18n/locales/en/common.json", "web/src/i18n/locales/kn/common.json", "web/src/i18n/locales/hi/common.json", "web/src

```

/App.build-error.test.tsx
(new),"web/src/components/IngestPanel.test.tsx"],"changeSpec":"PROBLEM (verified): App.tsx
onBuild (lines 145-150) catches a compile failure and does `const detail = err instanceof
ApiError && err.body ? parseDetail(err.body) : String(err); setBuild({ status: 'error', error:
detail })` so a 5xx-with-no-body shows raw 'HTTP 500'/an Error string, and a network throw
shows 'Error: Failed to fetch'. The ingest path already solved this with
lib/errors.ts::classifyError (5xx-no-body -> networkError 'Can't reach the engine. Is it
running?'; engine detail passes through). Reuse it for Build.\n\nNAMESPACE DECISION
(RECOMMENDED): introduce a SHARED `errors.*` i18n namespace for the friendly code->message
strings, and migrate the ingest copy onto it, because the IngestErrorCode set (invalidUri |
networkError | emptyResult | timeout | engine) is already flow-agnostic and Build reuses the
SAME codes. Reusing the `ingest.*` keys from a Build context would be semantically wrong (e.g.
'ingest.networkError' shown during a compile). So: add an `errors` block keyed by code, point
BOTH flows at it. (Alternative ? leave ingest.* alone and add build.* duplicates ? REJECTED:
duplicates the same 3 messages x3 locales and drifts.)\n\nEXACT onBuild CHANGE
(App.tsx:118-151): import `classifyError` (line 13 already imports parseDetail from ./lib/errors
? extend to `import { classifyError } from './lib/errors')`. In the catch block replace the
detail computation with:\n const e = classifyError(err);\n const msg = e.code === 'engine' ?
(e.detail ?? t('errors.engine')) : t(`errors.${e.code}`);\n setBuild({ status: 'error', error:
msg });\nKeep the existing `status`/log line (it already logs status + String(err) for
observability). Note: Build can only hit networkError | engine (it never produces
invalidUri/emptyResult/timeout), but mapping by code is harmless and future-proof; the floor-
block uses build.noVideo separately (App.tsx:123-130) ? unchanged.\n\nUNIFY IngestPanel
(RECOMMENDED, same PR): IngestPanel.tsx:30-35 builds errorMsg via
`t('ingest.${phase.error.code}')` and `t('ingest.failed', {error:''})`. Repoint to the shared
namespace: engine -> `phase.error.detail ?? t('errors.engine')`; else
`t('errors.${phase.error.code}')`. Then REMOVE the now-dead
ingest.{invalidUri,networkError,emptyResult,timeout,failed} keys from all three catalogs (their
text moves into errors.*). Keep
ingest.{title,intro,uriPlaceholder,...,working,progress,retry,cancel,submit,submitting,hint} as-
is (those are panel chrome, not error codes).\n\nLib note: classifyError already returns the
right thing for Build (it is generic over ApiError); no change to errors.ts needed.", "newI18nKey
s":["errors.invalidUri","errors.networkError","errors.emptyResult","errors.timeout","errors.engi
ne"],"testPlan":"1. NEW App.build-error.test.tsx (@vitest-environment jsdom, mirror
IngestPanel.test.tsx harness): render <App/> with ?video set (stub window.location.search or
seed via the IngestPanel onLoaded path), stub fetch so /api/compile returns (a) a bare 500 with
no body -> assert the compile-error testid text matches /can't reach the engine/i (the friendly
networkError copy), and (b) a 400 with {detail:'selected rallies all below min_shot_count'} ->
assert the engine detail passes through verbatim. Use data-testid='compile-error'
(CompileResult.tsx:64). 2. Update IngestPanel.test.tsx assertions that currently match the
ingest.* network copy to the (identical) errors.networkError text ? text is unchanged so the
/can't reach the engine/i matcher at line 90 still passes; just confirm green after the key
move. 3. errors.test.ts already pins classifyError ? no change, but it proves the mapping Build
now reuses. 4. hygiene key-parity test will FAIL loudly if errors.* isn't added to all of
en/kn/hi (that's the safety net) ? add all three. 5. `npm run test && npm run typecheck && npm
run build` green.", "gotchas":["KEY-PARITY CI (hygiene.test.ts) enforces en==kn==hi EXACTLY:
adding errors.* to en alone OR removing ingest.{networkError,...} from only one locale will turn
the suite red. Add the new errors.* and delete the migrated ingest.* keys in ALL THREE catalogs
in the same commit.", "kn/hi are AI-drafted (B5/B6, native review pending per the doc) ? the
moved error strings already have kn/hi translations under ingest.*; relocate those existing
translations into errors.* rather than re-drafting, so no quality regression and no new Gemini
draft is needed.", "Build's catch sets error:string into BuildState (CompileResult renders
build.error raw) ? so the localized message must be resolved to a STRING in App.tsx (do the t()
in onBuild), NOT passed as a key; CompileResult stays prose-agnostic. Keep it that way.", "The
engine-detail passthrough is intentional and honest (the engine's min_shot_count 400 is the most
specific message) ? do NOT translate engine.detail; only the code-based fallbacks are localized.
This matches errors.ts's existing design comment.", "onBuild already computes `status` for
logging from err; keep that log line intact for observability (current-state $observability
emphasis)."], "effort":"medium", "gated":"No (actionable)", {"id":"A8", "title":"Background-
processing reassurance + verdict/status as icon+text chips (never icon-only) (SPA)", "filesToTouc
h":["web/src/components/IngestPanel.tsx", "web/src/components/CompileResult.tsx", "web/src/componen
ts/SelectionFooter.tsx", "web/src/i18n/locales/en/common.json", "web/src/i18n/locales/kn/common.j
son", "web/src/i18n/locales/hi/common.json", "web/src/components/CompileResult.test.tsx
(new)", "web/src/components/IngestPanel.test.tsx"],"changeSpec":"GOAL (doc §4 lever 'Background-
processing reassurance + status as icon chips', medium/small; §3 HCI4D 'icon + text, never icon-

```

only', 'honest expectations on slow uplinks'): set honest expectations during the two long waits (ingest processing + reel build) and render success/failure verdicts as icon+TEXT chips.\n\n1) INGEST REASSURANCE (IngestPanel.tsx:74-85): the live-status block already shows a Spinner + `ingest.working` (and `ingest.progress` when polling). ADD a reassurance line beneath it: a small muted <p> with `t('ingest.reassure')` shown whenever `busy` ? copy that the job keeps running on the engine and they can leave/return via the video link (mirrors the honest ingest.timeout copy). Keep aria-live='polite' on the existing container; the reassurance is static so render it OUTSIDE the aria-live region (or it'll re-announce) ? put it as a sibling <p> after the status div.\n\n2) INGEST STATUS AS ICON+TEXT CHIP: the failure branch (IngestPanel.tsx:86-97) is already icon-free text ? fine. For the working state, the Spinner is aria-hidden + paired with text (good, already icon+text). No icon-only anywhere ? verify and leave.\n\n3) BUILD VERDICT CHIPS (CompileResult.tsx:31-43): the header currently shows `result.ready`/`result.failed` as plain text. Convert to an icon+text chip: a small rounded-full chip with an emoji/inline-SVG status glyph (aria-hidden) + the existing localized text ? green check chip for done, red chip for error. CRITICAL (doc rule): the text label MUST remain (never icon-only); the glyph is decorative (aria-hidden='true'). Reuse the existing color scheme (border-green/30 bg-green/5 vs border-red-200 bg-red-50 already on the <section>). Keep data-testid='compile-result' and the dismiss button.\n\n4) BUILD IN-FLIGHT REASSURANCE: CompileResult returns null while status==='building' (line 21) and the only building affordance is the footer button label `footer.building` (SelectionFooter.tsx:76). ADD a brief reassurance: render a small building chip in CompileResult when status==='building' (remove the early `|| build.status==='building` return; render a 'Stitching your reel?' + `result.buildingHint` line with a spinner) so the user sees progress in the main column, not only the sticky footer. Keep the footer button's disabled/Building? state. Use the same Spinner pattern as IngestPanel (or lift a shared <Spinner> ? optional, can stay duplicated for this small PR).\n\nFILE ANCHORS: IngestPanel.tsx:74-98 (status region), CompileResult.tsx:19-69 (verdict header + the building early-return at :21), SelectionFooter.tsx:70-77 (building button label ? leave label, it's the footer mirror).", "newI18nKeys": ["ingest.reassure", "result.building", "result.buildingHint", "result.statusDoneLabel", "result.statusFailedLabel"], "testPlan": "1. NEW CompileResult.test.tsx (@vitest-environment jsdom, mirror IngestPanel.test.tsx harness, mount real i18n at 'en'):\nrender with build={status:'building'} -> assert the building reassurance text (result.buildingHint) is visible AND a status label is present (icon+text, not icon-only).\nrender with status:'done' (url/filename) -> assert the verdict chip text result.ready is present alongside the (aria-hidden) glyph and the <video data-testid='reel-video'> + download link render.\nrender with status:'error' -> assert result.failed chip text + the error string render.\n2. Add an a11y assertion: the status glyph element has aria-hidden='true' and the human-readable label is queryable by text (proves never icon-only). 3. IngestPanel.test.tsx: add a case asserting that while polling, ingest.reassure text is visible; confirm it is NOT inside the aria-live node (query the sibling). 4. hygiene key-parity will fail unless the 5 new keys land in en+kn+hi ? add all three. 5. `npm run test && npm run typecheck && npm run build` green.", "gotchas": ["KEY-PARITY (hygiene.test.ts) is strict en==kn==hi ? add all 5 new keys to every catalog or CI goes red.", "Removing CompileResult's `|| build.status==='building` early return (line 21) changes when the section mounts ? ensure status:'idle' STILL returns null (only building should now render). Guard explicitly: `if (build.status === 'idle') return null;`.", "Reassurance copy must stay HONEST (doc §6, current-state launch-quality bar): the engine reports coarse phase WORDS not a %, so do NOT imply a percentage/ETA ? say 'this can take a few minutes; it keeps running if you leave', echoing the existing ingest.intro/ingest.timeout tone.", "Do NOT make any chip icon-only ? the doc's 'icon + text, never icon-only' is a hard rule; the glyph is aria-hidden decoration and the localized label is the real content.", "aria-live re-announcement: the existing ingest status div is aria-live='polite'; placing static reassurance inside it would re-read on every poll tick. Render reassurance as a sibling outside the live region.", "kn/hi are AI-drafted (native review pending) ? draft the 5 new keys for kn/hi too (consistent with B5/B6), do not leave them en-only (parity) and do not invent a native-review claim.", "effort": "small", "gated": "No (actionable)", {"id": "T4", "title": "In-UI local file upload ? wire SPA to existing chunked-upload endpoints, then hand the server path to /api/process", "filesToTouch": ["C:/Users/avidu/Projects/khelsutra/web/src/api/client.ts", "C:/Users/avidu/Projects/khelsutra/web/src/api/types.ts", "C:/Users/avidu/Projects/khelsutra/web/src/components/IngestPanel.tsx", "C:/Users/avidu/Projects/khelsutra/web/src/hooks/useIngestJob.ts", "C:/Users/avidu/Projects/khelsutra/web/src/lib/errors.ts", "C:/Users/avidu/Projects/khelsutra/web/src/i18n/locales/en/common.json", "C:/Users/avidu/Projects/khelsutra/web/src/i18n/locales/hi/common.json", "C:/Users/avidu/Projects/khelsutra/web/src/i18n/locales/kn/common.json", "C:/Users/avidu/Projects/khelsutra/web/src/api/client.test.ts", "C:/Users/avidu/Projects/khelsutra/web/src/components/IngestPanel.test.tsx"], "changeSpec": "KEYSTONE. The engine already exposes the full chunked-upload flow (backend/api/uploads.py): POST /api/upload/init {filename,total_size_bytes?} -> {upload_id,max_part_mb, next_part}; PUT /api/upload/{id}/part/{index} with RAW BYTES body (sequential

0,1,2...) -> {received_parts,received_bytes}; POST /api/upload/{id}/complete {total_parts} -> {path (ABSOLUTE server path), video_id, size_bytes}; DELETE /api/upload/{id} to abort. The bundled engine app.js never calls these and the SPA has no file UI ? this ticket is CLIENT-WIRING ONLY, ZERO engine change.\n\nNET-NEW (client) in api/client.ts: add upload helpers that DO NOT go through the existing request() helper ? request() hardcodes content-type:application/json and JSON.stringify(body), which corrupts a raw-bytes part PUT. (1) uploadInit(file, cid): POST /api/upload/init with JSON {filename:file.name, total_size_bytes:file.size} via request(); returns {upload_id,max_part_mb,next_part}. (2) uploadPart(uploadId, index, blob, cid): a BESPOKE fetch (not request()) -> PUT /api/upload/{id}/part/{index}, body: the Blob slice, headers: {content-type:'application/octet-stream','x-correlation-id':cid}; reuse the same !res.ok -> ApiError mapping + log lines as request() (factor a small _rawRequest, or inline). (3) uploadComplete(uploadId, totalParts, cid): POST /api/upload/{id}/complete via request(); returns {path,video_id,...}. (4) uploadAbort(uploadId, cid): DELETE via request(). Add a high-level uploadFile(file, {onProgress, signal, cid}) orchestrator: init -> read max_part_mb from the init response (DO NOT hardcode 95 ? the engine returns it from security.max_part_mb, default 95) -> slice file into ceil(size/(max_part_mb*1024*1024)) parts via file.slice(off, off+partBytes) -> PUT each SEQUENTIALLY (await each before the next; the engine 409s out-of-order parts) -> call onProgress({sentBytes,totalBytes,partIndex,partCount}) after each -> complete(partCount) -> return {path,videoId}. Honor an AbortSignal: on abort/unmount, stop the loop and fire uploadAbort(uploadId) so the server's .partial file is reaped (uploads are in-process, a restart also reaps them). Add the response types to api/types.ts (UploadInitResponse, UploadPartResponse, UploadCompleteResponse) mirroring uploads.py exactly.\n\nNET-NEW (UI) in IngestPanel.tsx: add a SOURCE TOGGLE above the existing form ? two modes, 'Upload a file' and 'Bucket / path' (the existing URI input, unchanged = REUSED). In upload mode render <input type=file accept='video/mp4,video/quicktime,.mp4,.mov'> (the engine allows only mp4/mov per security.allowed_upload_extensions; surface that). On file pick, show name + human size; on Upload click, call client.uploadFile with an onProgress that drives a DETERMINATE bar (sentBytes/totalBytes %) ? this is the one place we have a real percentage, unlike the indeterminate process poll. A Cancel button aborts via the AbortSignal. The existing Spinner/aria-live status block is REUSED for the post-upload phase.\n\nJOINING THE EXISTING PATH (reused, do not rebuild): uploadFile resolves to {path,videoId}; feed path into the EXISTING submit() of useIngestJob (which POSTs /api/process {video_path} and polls /api/jobs/{id}). useIngestJob.submit currently takes a string and trims/empty-checks for a URI ? extend it minimally to accept either a typed URI or an upload-produced absolute server path (the absolute path is just another video_path string; no new submit code path needed beyond not rejecting it). From there the UNCHANGED

poll->done->onLoaded->App.setVideoId->listRallies->RallyGrid->compile->highlightUrl download chain runs verbatim. So the join point is exactly: upload (net-new) -> path -> existing submit(path) -> existing everything.\n\nERRORS (errors.ts, reused classifier): the upload PUT/complete throw ApiError on 400 (bad ext / unsafe filename via safe_output_filename), 413 (part>max_part_mb or total>max_upload_mb -> 'Upload aborted, re-init'), 409 (out-of-order/part-count mismatch). Add upload-specific IngestErrorCode entries (uploadTooLarge, uploadBadType, uploadFailed) OR route 413/400 through the existing 'engine' code so the verbatim FastAPI detail shows. Keep the hook i18n-free (codes, not prose) per the existing convention.\n\nMAX-PART SIZE: never hardcode; the value is authoritative from the init response (engine security.max_part_mb default 95MB, chosen because free-tier edge proxies cap bodies ~100MB ? directly relevant to the T2 Cloudflare deploy). PROGRESS: determinate during upload (bytes), then the existing indeterminate phase-word spinner during process.", "newI18nKeys":["ingest.sourceUpload","ingest.sourceUri","ingest.pickFile","ingest.fileSelected","ingest.uploading","ingest.uploadProgress","ingest.uploadCancel","ingest.uploadTooLarge","ingest.uploadBadType","ingest.uploadFailed","ingest.allowedTypesHint"],"testPlan":"Mirror existing vitest conventions. (1) api/client.test.ts (extend, pattern from current mockFetch/lastCall): assert uploadInit POSTs JSON to /api/upload/init; uploadPart PUTs to /api/upload/{id}/part/{i} with content-type octet-stream and a NON-json body (assert request() was bypassed ? body is the blob, not JSON.stringify); uploadComplete POSTs {total_parts}; uploadFile slices a fake File into the right part count from a mocked max_part_mb and PUTs sequentially in order 0,1,2; onProgress called per part; an aborted signal triggers a DELETE /api/upload/{id}; a 413 on a part throws ApiError with the engine detail. (2) IngestPanel.test.tsx (extend, jsdom): toggle to upload mode, pick a File (new File([bytes],'m.mp4',{type:'video/mp4'})), click Upload, mock the full init/part/complete fetch sequence + the subsequent /api/process+/api/jobs done sequence, assert onLoaded gets the video_id; assert a determinate progress text appears; assert a 400 bad-extension shows the engine detail. (3) i18n hygiene.test.ts auto-enforces key-parity across en/hi/kn for the new keys (no test edit needed, but all three catalogs MUST be updated or CI red). Browser-verify (L1, Claude Preview): npm run dev with the engine stub (python -m backend.main --server,

detector_impl=stub, skip_ai_handoff)), pick a small real mp4, watch the bar fill, then the phase poll, then the rally grid load.", "gotchas": ["request() in client.ts forces content-type: application/json + JSON.stringify ? a part PUT MUST bypass it or the raw bytes get JSON-wrapped and the assembled file is corrupt. This is the single biggest implementation trap.", "Parts are STRICTLY sequential ? the engine 409s if index != expected. Do not parallelize part PUTs; await each.", "max_part_mb comes from the init RESPONSE (engine config), not a client constant; the 95MB default exists specifically so parts stay under the ~100MB edge-proxy body cap that T2's Cloudflare deploy enforces.", "upload_complete returns an ABSOLUTE path; /api/process must accept it. In single-user mode (allowed_video_root=None) it's fine, but the T2 hosted posture sets security.allowed_video_root, and it MUST contain security.upload_dir or /api/process 400s the uploaded path (warned in SecurityConfig docstring, models.py:287). Flag this cross-ticket to T2.", "Upload state is in-process (uploads.py module dict) ? a server restart aborts partial uploads; the client must surface a clean re-init, not hang. The hook's existing cancel/supersede idiom covers most of this.", "ALL THREE locale catalogs (en/hi/kn) must get the new keys in the same PR or the i18n key-parity test (hygiene.test.ts) goes red. hi/kn can be AI-drafted (native review pending, per existing convention).", "accept= on the file input is a hint only; the engine is the real gate (allowed_upload_extensions=[mp4,mov]). Don't claim other formats work."], "effort": "medium", "gated": "No (actionable)", {"id": "T2", "title": "Droplet appliance shell ? serve the built SPA + reverse-proxy /api to the CPU engine behind Cloudflare Access (Gemini/CPU, no GPU)", "filesToTouch": ["C:/Users/avidu/Projects/khelsutra/site/scripts/provision-vm.sh", "C:/Users/avidu/Projects/khelsutra/site/scripts/teardown-vm.sh", "C:/Users/avidu/Projects/khelsutra/web/vite.config.ts", "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md", "C:/Users/avidu/Projects/khelsutra/deploy/appliance/Caddyfile (net-new)", "C:/Users/avidu/Projects/khelsutra/deploy/appliance/khelsutra-engine.service (net-new)", "C:/Users/avidu/Projects/khelsutra/deploy/appliance/provision-appliance.sh (net-new)", "C:/Users/avidu/Projects/khelsutra/deploy/appliance/README.md (net-new)"], "changeSpec": "Stand up the existing SPA + engine as ONE hosted, shareable origin on the CPU droplet 168.144.126.176 (region blr1, CPU-only, NO GPU), engine on Gemini or stub. The auth boundary is the EDGE, not the app: the engine has zero app-auth and CORS '*' and is HARD-bound to 127.0.0.1:8000 (verified backend/main.py:756 uvicorn.run(host='127.0.0.1')); CORS at main.py:62 default ['*']). Safety holds iff all three: engine on 127.0.0.1 + firewall blocks inbound 8000 + the edge gate is the sole ingress.\n\nTOPOLOGY (LOCAL_APPLIANCE_ARCHITECTURE.md §4.1): public -> Cloudflare Access (edge auth) -> cloudflared Tunnel (outbound-only, no inbound ports) -> Caddy/static host on the box -> serves the built web/ SPA at / and /assets/*, reverse-proxies /api/* and /api/highlights/* -> 127.0.0.1:8000 engine. ONE origin to the browser, so the engine's CORS '*' is irrelevant (D1). The marketing site stays a SEPARATE systemd unit (khelsutra-site, :8787) on a SEPARATE subdomain ? do not disturb it.\n\n(1) BUILD THE SPA: cd web && npm ci && npm run build -> web/dist (Vite). The SPA already makes same-origin relative /api/* calls (vite.config.ts comment confirms prod reverse-proxy model) so NO build-time origin config needed; VITE_ENGINE_ORIGIN override exists if ever required. Ship dist/ to the box (git archive HEAD:web pattern from provision-vm.sh, or build locally + scp dist).\n\n(2) STATIC HOST + REVERSE PROXY (net-new deploy/appliance/Caddyfile): Caddy is the recommended choice (D2) ? in the cloudflared-Tunnel model Caddy listens on a local port and the Tunnel terminates TLS. Caddyfile: serve web/dist as the file root with try_files {path} /index.html (SPA fallback for client routing); reverse_proxy /api/* 127.0.0.1:8000. Alternative reuse: the existing site server.js (PORTABILITY.md) serves public/ but has no /api proxy ? Caddy is cleaner for the proxy; document both. NOTE the SPA is air-gapped/self-contained (hygiene.test.ts asserts no CDN), so static serving is trivial ? all assets are bundled.\n\n(3) WIRE /api -> 127.0.0.1:8000: engine launched via python -m backend.main --server --port 8000 (binds 127.0.0.1 by construction). Caddy proxies /api -> :8000. Firewall (ufw) MUST block inbound 8000 AND the shared-droplet test ports 9000/9001 (MinIO) + 9023 (GCS emu) ? only the Tunnel/Caddy ingress is open. This is the T3 posture invariant (raw :8000 refused from outside) ? T3 is the test that verifies it; T2 must SET it up correctly.\n\n(4) CLOUDFLARE ACCESS (high level, D2/§4.4): install cloudflared on the box; create a named Tunnel; route the SPA hostname (e.g. studio.khelsutra.guru) through it to the local Caddy port; in the Cloudflare dashboard create a Cloudflare Access application over that hostname with an email-OTP policy (owner + invited alpha emails). Outbound-only Tunnel = no inbound ports opened, matches the two-host design. Engine optionally trusts Cf-Access-Authenticated-User-Email / verifies the Cf-Access-Jwt-Assertion (engine M9 edge_auth, SecurityConfig.edge_auth_enabled default-OFF, models.py:303) ? for the single-tenant alpha leave engine edge_auth OFF and rely on the gate; document turning it on for tenant tagging.\n\n(5) ENGINE DEPLOY: deploy the #281/#289-fixed engine (the double-module async-job drain fix is in master per the BHI git log: 41d45fd 'drain async jobs under python -m backend.main'). Provide GEMINI_API_KEY via env (.env / systemd Environment=, NEVER in repo) and install google-cloud-storage so gs:// bucket ingest works (backend/storage/gcs.py imports it;

the bucket-URI ingest CUJ T12 is already shipped). Engine deps: pip install the engine requirements incl. google-cloud-storage; ffmpeg present on the box. Net-new systemd unit khelsutra-engine.service: WorkingDirectory=engine repo, ExecStart=python -m backend.main --server --port 8000, Environment for GEMINI_API_KEY + RALLY_CORS_ALLOW_ORIGINS (can stay default since single-origin), Restart=always. Set security.allowed_video_root to CONTAIN security.upload_dir (output/uploads) so T4's uploaded absolute paths pass /api/process validation (cross-ticket with T4 + the SecurityConfig.allowed_video_root warning at models.py:287).

(6) KEY ROTATION (go-live, \$4.4 / HOSTING_PLAN go-live checklist): the GEMINI_API_KEY shared in chat MUST be rotated before any non-owner hits the box; set the new key on-box only (env/systemd), never paste in chat or commit. Same for any GCS service-account creds (boto/google default chain on-box, not in config). Least-privilege; rotate-after.

REUSED vs NET-NEW: REUSED = the provision-vm.sh/teardown-vm.sh provider-agnostic pattern (git archive private-repo-safe push, Node install, systemd unit, ufw, verify-it-serves loop, DB export/restore for clean destroy->resume) ? adapt it into a sibling provision-appliance.sh that does the SAME for web/dist + Caddy + the engine unit + cloudflared; REUSED = the existing khelsutra-site systemd/firewall precedent; REUSED = the SPA itself (built, unchanged) + its same-origin /api design + the engine's already-shipped /api/health (main.py:148) for a readiness probe. NET-NEW = Caddyfile, khelsutra-engine.service, cloudflared Tunnel + Cloudflare Access config (dashboard, owner step), the engine pip/env bootstrap, the firewall rules for 8000/900x, and provision-appliance.sh orchestrating it.

OWNER STEPS (gated): (a) create the Cloudflare Tunnel + Access application + email policy in the CF dashboard (account-level, not scriptable safely); (b) rotate GEMINI_API_KEY and place it on-box; (c) authenticate cloudflared on-box (cloudflared login) so no token enters chat/repo; (d) approve creating/keeping the paid CPU droplet (cost rule: a stopped VM still bills disk ? DELETE to \$0 per the owner GCP-VM rule, though this DO droplet is a shared testbed). Provisioning creds must be entered on-box (doctl auth init interactive), never in chat ? per the LOCAL_APPLIANCE_ARCHITECTURE \$6 credential-leak risk row.

"newI18nKeys": [], "testPlan": "Infra ticket ? verification is operational + a posture assertion, not unit tests. (1) Reuse provision-vm.sh's verify-it-serves idiom: after provision-appliance.sh, curl http://127.0.0.1:<caddy-port>/ returns the SPA index (assert an index marker) and curl http://127.0.0.1:8000/api/health returns {status:ok} through the proxy at /api/health. (2) THROUGH THE GATE (L3, LOCAL_APPLIANCE_ARCHITECTURE \$8): from outside, the Cloudflare Access login is required before the SPA loads; once authed, run CUJ-1 end-to-end on the droplet with engine stub (key-free) then gemini (real, a SHORT clip to bound cost) ? ingest (upload via T4 + a gs:// URI via T12) -> poll -> rally pick -> compile -> download. (3) EXPOSURE POSTURE TEST (this is T3, but T2 must pass it): from an external host, raw http://168.144.126.176:8000/ and :9000/:9001/:9023 must be REFUSED/timeout (ufw blocks them) ? assert, don't assume. (4) Confirm the marketing site (khelsutra-site :8787) still serves ? the two units are independent. (5) Inspect engine + Caddy logs during the live CUJ replay (Observability \$5): the job_id correlation threads ingest->process->jobs->compile.", "gotchas": ["The engine is HARD-bound to 127.0.0.1 (main.py:756) ? do NOT 'fix' it to 0.0.0.0 to make the proxy work; the proxy talks to 127.0.0.1:8000 from the same box. Binding 0.0.0.0 would break the tenancy invariant (CORS '*' + zero auth = open box).", "Safety is THREE conditions ANDed (127.0.0.1 bind + firewall blocks 8000/900x + gate up). If any slips, the box is open. The shared droplet ALSO runs MinIO 9000/9001 + GCS emu 9023 ? firewall those too, not just 8000.", "Rotate the chat-shared GEMINI_API_KEY BEFORE non-owner exposure; set the new key on-box only. Never paste keys/tokens in chat or commit them (the already-rotated DO Spaces key incident is the precedent).", "Set security.allowed_video_root to CONTAIN upload_dir or T4's uploaded absolute paths get 400'd by /api/process (models.py:287 warning). Coordinate with T4.", "Run T11 (retention sweep) BEFORE inviting any non-owner ? output/ + uploads/ hold raw originals and prior-session reels that would otherwise be visible to every authed alpha user (single-tenant, no per-user scoping).", "No GPU on this box: only gemini (real, cloud ? consent copy must say frames go to Google) or stub (CPU mock ? must be labelled 'not real detection') or motion (low-quality). Do NOT present native WASB as available here.", "cloudflared Tunnel is outbound-only ? do NOT open inbound 443/80 on the box for it; that's the whole point (no inbound ports). Caddy listens locally; the Tunnel reaches it.", "Branch-safety: this touches the engine repo's deploy + the khelsutra repo. Branch from origin/master in EACH, re-verify HEAD before commit, stage explicit paths (shared-checkout collision risk per CLAUDE.md).", "effort": "large", "gated": "Owner (decision)", {"id": "T11", "title": "Retention sweep for output/ + uploads/ before non-owner exposure", "filesToTouch": ["C:/Users/avidu/Projects/badminton-highlight-indexer/tools/cleanup_caches.py", "C:/Users/avidu/Projects/badminton-highlight-indexer/tools/retention_sweep.py (net-new, or extend cleanup_caches.py)", "C:/Users/avidu/Projects/badminton-highlight-indexer/tests/tools/test_retention_sweep.py (net-new)", "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md"], "changeSpec": "Before any non-owner hits the hosted box (T2), the single-tenant alpha has NO per-user scoping ? output/ (reels, the SQLite DB, reports) and

security.upload_dir (default output/uploads, raw uploaded originals) are GLOBAL and visible to every authed user via /api/highlights and the rally history. LOCAL_APPLIANCE_ARCHITECTURE \$6 risk row: 'output/+uploads/ originals never deleted'; the DoD requires a retention sweep in place. This ticket adds a retention sweep for raw originals + stale outputs.

The existing tools/cleanup_caches.py is a CLOSE precedent and partly REUSABLE: it has the dry-run-by-default discipline, a pure unit-tested classify()/plan_deletions() split, --apply, --older-than (age cutoff), --keep-video (protect by id substring), human() sizes, and a scan_windows(output_dir) that walks output/ + output/wasb. BUT it is scoped to REGENERABLE pipeline caches (frames/staged/handoff/proxies) and EXPLICITLY KEEPS .mp4 reels, .db, and .csv as durable (classify() returns keep for highlight reels + database). Retention is a DIFFERENT policy: it must purge RAW UPLOADED ORIGINALS (security.upload_dir contents, incl. the assembled files and any .partial-* leftovers from aborted/restart-killed uploads) and OPTIONALLY age-out finished reels/outputs after a window ? i.e. it deletes things cleanup_caches deliberately preserves.

IMPLEMENTATION (prefer a NEW tools/retention_sweep.py reusing cleanup_caches's helpers ? human(), the Entry dataclass shape, the dry-run/--apply/--older-than/_report scaffolding ? rather than overloading the cache classifier with a contradictory policy):

- (1) Read security.upload_dir + output.output_dir from the typed config (backend.config load_app_config) rather than hardcoding 'output' ? cleanup_caches hardcodes --output-dir default 'output'; retention should read the actual configured dirs so it matches what the engine writes.
- (2) Targets: ALL files under upload_dir (the raw originals + .partial-* aborted-upload leftovers) older than --upload-retention-days (default e.g. 1 day, or 0 = delete after compile ? see policy gate below); stale reels/reports under output_dir older than --output-retention-days.
- (3) Dry-run by DEFAULT (print PURGE vs KEEP + reclaimable bytes), --apply to delete ? mirror cleanup_caches exactly.
- (4) --keep-video to protect an id; never delete the SQLite DB (rally history) or an in-flight upload's .partial that is younger than the cutoff.
- (5) Loud: print every deleted path; never silent-drop.

POLICY (OWNER-GATED, LOCAL_APPLIANCE_ARCHITECTURE \$10 Q2 + risk table): the exact defaults are an owner decision ? auto-delete originals after compile? keep proxies+intervals only? what window? This ticket builds the MECHANISM with a conservative default (dry-run; a documented retention window) and surfaces the policy choice to the owner; the default flip to --apply on a schedule is a separate, owner-approved step. Wire it as a one-shot CLI the appliance runbook runs before exposure (and optionally a cron/systemd-timer on the box ? note that, but installing a destructive timer is owner-gated).

DOC: add a retention section to LOCAL_APPLIANCE_ARCHITECTURE (or update the \$6 risk row from 'never deleted' to 'swept by tools/retention_sweep.py') honestly ? only after the mechanism lands.

```

{
  "newI18nKeys": [],
  "testPlan": "Mirror cleanup_caches's pure-function test pattern (the BHI run_all_tests.py offline suite). (1) Unit-test the pure plan: given a synthetic set of Entry-like rows (an uploaded match.mp4 2 days old, a .partial-abc aborted leftover, a fresh upload 1 min old, a reel.mp4, the access .db), assert plan: stale upload + old .partial are PURGE; fresh upload + .db are KEEP; --keep-video protects a matching id; --older-than/window cutoff respected; default (no --apply) deletes nothing. (2) Assert it reads upload_dir + output_dir from a mock AppConfig, not a hardcoded 'output'. (3) A tmp_path integration test: create real files under a temp upload_dir/output_dir, run with --apply, assert the stale raw original + .partial are gone and the .db + fresh upload survive. (4) Run python run_all_tests.py to confirm the offline suite stays green + coverage at/above floor. Browser-verify N/A (pure tooling); the owner runs it on-box before exposure as the operational check.",
  "gotchas": [
    "Do NOT reuse cleanup_caches.classify() as-is ? it KEEPS .mp4 reels + .db + .csv (durable) and only purges regenerable caches; retention must purge raw uploaded ORIGINALS, the opposite policy. A new module reusing its scaffolding is cleaner than contradicting the classifier.",
    "NEVER delete the SQLite DB (rally history / jobs) or a .partial younger than the cutoff (an in-flight upload ? uploads are in-process, the file is being appended). Aborted/orphaned .partial-* from a server restart ARE safe to reap.",
    "Read security.upload_dir + output.output_dir from the typed config ? they are configurable (models.py:289 upload_dir default output/uploads; M3 made output base configurable). Hardcoding 'output' will miss a relocated dir.",
    "The exact retention defaults (delete-after-compile? keep proxies+intervals? window length?) are OWNER-GATED (LOCAL_APPLIANCE $10 Q2) ? ship the mechanism dry-run-default + surface the choice; do NOT auto-enable a destructive scheduled sweep without owner sign-off.",
    "Single-tenant alpha = global visibility: without this sweep, every invited alpha user sees every prior session's raw originals + reels via /api/highlights and the history. Run it (and confirm) BEFORE the T2 gate opens to non-owners ? this is a launch blocker, not a nicety.",
    "Keep dry-run the default + --apply opt-in (cleanup_caches discipline); a destructive default has bitten this project before."
  ],
  "effort": "small",
  "gated": "Owner (decision)"
}

```

Produce: the recommended execution ORDER across all tickets; cross-item coordination (which share App.tsx/catalogs and must serialize); a crisp North-Star summary (the cheapest path to alpha-shareable: T4 upload ? T2 droplet+Cloudflare-Access+Gemini/CPU ? T11); and risks. Return

structured JSON.

2. assistant (2026-06-21T14:39:47.152Z)

I'll analyze these tickets against the actual codebase to produce an accurate handoff plan. Let me start by pulling the latest and verifying the key claims about the code.

3. user (2026-06-21T14:39:50.941Z)

```
600ddbc feat(web): A1 ? plain-language copy pass on the en catalog (de-jargon)
a20002f docs: propose Multilingual Expansion ? 7 new locales (th/id/da/es/zh-Hans/ja/ko) (#58)
a0475bc docs: hygiene refresh ? sync trackers to shipped reality (#59)
---BRANCH---
feat/a1-plain-language-copy
```

4. user (2026-06-21T14:39:52.772Z)

```
a1bec6f fix(dryrun): install the watermark font in bootstrap + track the UI-trigger asks
7835f34 feat(gcp): always-install Ops Agent on GPU VMs + a cloud-GPU dry-run guide (#291)
aac7633 feat(serving): M9-auth ? Cloudflare Access edge-auth + per-tenant owner_id (default-OFF)
(#290)
---BRANCH---
fix/dryrun-guide-font-and-ui-tracking
```

5. assistant (2026-06-21T14:39:55.809Z)

Let me verify the key code claims. I'll check the SPA structure, the engine upload endpoints, the site structure, and the i18n setup in parallel.

6. user (2026-06-21T14:39:59.495Z)

```
Exit code 2
=== web/src/i18n ===
web/src/i18n/:
total 28
drwxr-xr-x 1 avidu 197609  0 Jun 21 15:45 .
drwxr-xr-x 1 avidu 197609  0 Jun 21 15:45 ..
-rw-r--r-- 1 avidu 197609 4467 Jun 21 15:45 hygiene.test.ts
-rw-r--r-- 1 avidu 197609 7027 Jun 21 12:43 i18n.test.tsx
-rw-r--r-- 1 avidu 197609 2851 Jun 21 12:43 index.ts
drwxr-xr-x 1 avidu 197609  0 Jun 21 12:43 locales

web/src/i18n/locales/:
total 4
drwxr-xr-x 1 avidu 197609 0 Jun 21 12:43 .
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 ..
drwxr-xr-x 1 avidu 197609 0 Jun 21 20:09 en
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 hi
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 kn
=== top-level locales? ===
NO top-level locales/
=== site/public ===
total 164
drwxr-xr-x 1 avidu 197609  0 Jun 21 13:08 .
drwxr-xr-x 1 avidu 197609  0 Jun 21 12:12 ..
-rw-r--r-- 1 avidu 197609  6983 Jun 20 11:30 capture.html
-rw-r--r-- 1 avidu 197609 13754 Jun 21 13:08 flyer-hi.html
-rw-r--r-- 1 avidu 197609 14410 Jun 21 13:08 flyer-kn.html
-rw-r--r-- 1 avidu 197609 10938 Jun 21 13:08 flyer.html
drwxr-xr-x 1 avidu 197609  0 Jun 21 13:08 img
-rw-r--r-- 1 avidu 197609 43816 Jun 21 12:12 index.html
-rw-r--r-- 1 avidu 197609 40635 Jun 20 11:30 og.png
-rw-r--r-- 1 avidu 197609 15007 Jun 21 12:12 own-model.html
drwxr-xr-x 1 avidu 197609  0 Jun 20 23:27 video
```

```
=== site/src ===
assets.test.js
deploy-config.test.js
index.js
index.test.js
notify.js
server.test.js
store.js
ux.dom.test.js
=== site/scripts ===
droplet-smoke.sh
parity-check.sh
provision-vm.sh
teardown-vm.sh
=== deploy/ ===
```

7. user (2026-06-21T14:40:00.787Z)

```
=== web/src/components ===
CompileResult.tsx
IngestPanel.test.tsx
IngestPanel.tsx
LanguageSwitcher.tsx
QueryBar.examples.test.ts
QueryBar.tsx
RallyGrid.tsx
SelectionFooter.tsx
=== web/src/api ===
client.test.ts
client.ts
types.ts
=== web/src/lib ===
errors.test.ts
errors.ts
floor.test.ts
floor.ts
format.test.ts
format.ts
log.ts
query.test.ts
query.ts
selection.test.ts
selection.ts
=== web/src/hooks ===
useIngestJob.test.ts
useIngestJob.ts
useRallySelection.ts
```

8. assistant (2026-06-21T14:40:04.136Z)

Notable discrepancies already: `deploy/` is empty/missing, `server.js` lives where? site/src has `index.js` not `server.js`, and T2 references `site/scripts/provision-vm.sh` (exists). Let me verify the critical code claims in parallel.

9. user (2026-06-21T14:40:05.103Z)

```
1      // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2      //
3      // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4      //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5      //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6      //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7      //   - one negotiation order shared with the marketing site (B3);
```

```

8      // - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9      import i18n from "i18next";
10     import { initReactI18next } from "react-i18next";
11     import LanguageDetector from "i18next-browser-languagedetector";
12     import ICU from "i18next-icu";
13     import enCommon from "../locales/en/common.json";
14     import knCommon from "../locales/kn/common.json";
15     import hiCommon from "../locales/hi/common.json";
16
17     // Supported locales (D6): `en` + `kn` + `hi`. Anything else falls back to `en`.
18     export const SUPPORTED_LOCALES = ["en", "kn", "hi"] as const;
19     export type Locale = (typeof SUPPORTED_LOCALES)[number];
20     export const DEFAULT_LOCALE: Locale = "en";
21
22     // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
23     // visitor's language choice carries across both surfaces.
24     export const LANG_STORAGE_KEY = "khelsutra.lang";
25
26     // Catalogs are co-located here (D9). `kn` + `hi` shipped (B5/B6, AI-drafted ? native
review
27     // pending). A missing key in any locale still falls back to `en` ? the by-design
invariant.
28     const resources = {
29       en: { common: enCommon },
30       kn: { common: knCommon },
31       hi: { common: hiCommon },
32     } as const;
33
34     void i18n
35       .use(ICU)
36       .use(LanguageDetector)
37       .use(initReactI18next)
38       .init({
39         resources,
40         fallbackLng: DEFAULT_LOCALE,
41         supportedLngs: [...SUPPORTED_LOCALES],
42         nonExplicitSupportedLngs: true, // en-IN -> en
43         defaultNS: "common",
44         ns: ["common"],
45         interpolation: { escapeValue: false }, // React already escapes
46         returnNull: false,
47         // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
48         detection: {
49           order: ["localStorage", "querystring", "navigator"],
50           lookupLocalStorage: LANG_STORAGE_KEY,
51           lookupQuerystring: "lang",
52           caches: ["localStorage"],
53         },
54         react: { useSuspense: false },
55       });
56
57     // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
58     // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
59     function syncDocumentLang(lng: string) {
60       if (typeof document !== "undefined") {
61         document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
62       }
63     }
64     i18n.on("languageChanged", syncDocumentLang);
65     syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);

```

```
66
67     export default i18n;
68
```

10. user (2026-06-21T14:40:05.588Z)

```
1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };
10             if (j && typeof j.detail === "string") return j.detail;
11         } catch {
12             /* not JSON ? fall through to the raw body */
13         }
14         return body;
15     }
16
17     /** Stable, localizable failure categories surfaced by the ingest flow.
18     * `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19     export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21     export interface IngestError {
22         code: IngestErrorCode;
23         /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24         detail?: string;
25     }
26
27     /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28     * is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29     * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30     export function classifyError(err: unknown): IngestError {
31         if (err instanceof ApiError) {
32             if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable
33             if (err.body) {
34                 const detail = parseDetail(err.body);
35                 if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36             }
37             // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38             // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39             // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40             if (err.status >= 500) return { code: "networkError" };
41             return { code: "engine", detail: `HTTP ${err.status}` };
42         }
43         return { code: "engine", detail: String(err) };
44     }
45
```

11. user (2026-06-21T14:40:05.848Z)

```
1      // Typed client for the engine FastAPI. SAME-ORIGIN relative `/api/*` (the appliance
serves
2      // UI + engine behind one origin; in dev Vite proxies `/api` ? :8000). Every call
carries a
3      // correlation id (header + logs) so a journey is traceable end to end.
4      import { log, newCorrelationId } from "../lib/log";
5      import type { RallySegment, QueryResponse, CompileResponse, EngineConfig,
ProcessResponse, JobStatus } from "../types";
6
7      const BASE = "/api";
8
9      export class ApiError extends Error {
10         readonly cid?: string;
11         readonly status?: number;
12         readonly body?: string;
13         constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
14             super(message);
15             this.name = "ApiError";
16             this.cid = opts.cid;
17             this.status = opts.status;
18             this.body = opts.body;
19             if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
20         }
21     }
22
23     interface RequestOpts {
24         method?: string;
25         body?: string;
26         headers?: Record<string, string>;
27         /** Correlation id to thread through this call; auto-generated if absent. */
28         cid?: string;
29     }
30
31     async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
32         const cid = opts.cid ?? newCorrelationId();
33         const url = `${BASE}${path}`;
34         const method = opts.method ?? "GET";
35         log("info", "api.request", { cid, method, url });
36
37         let res: Response;
38         try {
39             res = await fetch(url, {
40                 method,
41                 body: opts.body,
42                 headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
43             });
44         } catch (err) {
45             // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
46             log("error", "api.network_error", { cid, method, url, error: String(err) });
47             throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
48         }
49
50         if (!res.ok) {
51             const body = await res.text().catch(() => "");
52             log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
53             throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
54         }
55
56         const data = (await res.json()) as T;
```

```

57     log("info", "api.response", { cid, method, url, status: res.status });
58     return data;
59 }
60
61 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
62     * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
63     * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
64     export async function processVideo(videoPath: string, cid?: string):
Promise<ProcessResponse> {
65         return request<ProcessResponse>("/process", {
66             method: "POST",
67             body: JSON.stringify({ video_path: videoPath }),
68             cid,
69         });
70     }
71
72 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
73 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
74     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
75 }
76
77 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
78 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
79     const data = await request<QueryResponse>("/query", {
80         method: "POST",
81         body: JSON.stringify({ video_id: videoId }),
82         cid,
83     });
84     return data.play_segments ?? [];
85 }
86
87 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
88 export async function compileReel(
89     videoId: string,
90     segmentIds: number[],
91     outputFilename: string,
92     cid?: string,
93 ): Promise<CompileResponse> {
94     return request<CompileResponse>("/compile", {
95         method: "POST",
96         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename }),
97         cid,
98     });
99 }
100
101 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
102 export function highlightUrl(videoId: string, filename: string): string {
103     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
104 }
105
106 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
107 export async function getConfig(cid?: string): Promise<EngineConfig> {
108     return request<EngineConfig>("/config", { method: "GET", cid });
109 }
110

```



```

111     /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
112     export function minShotCount(cfg: EngineConfig): number | undefined {
113         return cfg.stitching?.min_shot_count;
114     }
115

```

12. assistant (2026-06-21T14:40:09.733Z)

The client.ts confirms T4's key trap: `request()` hardcodes `content-type: application/json`. Now let me verify the engine upload endpoints, the App.tsx onBuild path, the site server (index.js vs server.js), and the SPA build/deploy assumptions.

13. user (2026-06-21T14:40:10.631Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """
15    import os
16    import threading
17    import uuid
18    from typing import Any, Dict, Optional
19
20    from fastapi import APIRouter, HTTPException, Request
21    from pydantic import BaseModel
22
23    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24    from backend.utils.paths import safe_output_filename
25
26    router = APIRouter()
27
28    _uploads: Dict[str, Dict[str, Any]] = {}
29    _uploads_lock = threading.Lock()
30
31
32    class UploadInitRequest(BaseModel):
33        filename: str
34        total_size_bytes: Optional[int] = None
35
36
37    class UploadCompleteRequest(BaseModel):
38        total_parts: int
39
40
41    def _upload_cfg():
42        # Resolved lazily (import inside the function) to read the LIVE startup config that
43        # main() sets on the module global, and to avoid a circular import at module load
44        # (main.py imports this router; this module must not import main at import time).
45        import backend.main as _main
46        sec = getattr(_main.global_config, "security", None)
47        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50        exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",

```

```

None) or ["mp4", "mov"]]]
51     return upload_dir, max_bytes, part_bytes, exts
52
53
54     def _abort_upload(upload_id: str) -> None:
55         with _uploads_lock:
56             st = _uploads.pop(upload_id, None)
57             if st:
58                 try:
59                     os.remove(st["tmp_path"])
60                 except OSError:
61                     pass
62
63
64     @router.post("/api/upload/init")
65     def upload_init(req: UploadInitRequest):
66         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68         try:
69             name = safe_output_filename(req.filename)
70         except ValueError as e:
71             raise HTTPException(status_code=400, detail=str(e)) from e
72         ext = os.path.splitext(name)[1].lower().lstrip(".")
73         if ext not in exts:
74             raise HTTPException(status_code=400,
75                                 detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
76         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
77             raise HTTPException(status_code=413,
78                                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79         os.makedirs(upload_dir, exist_ok=True)
80         upload_id = uuid.uuid4().hex
81         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82         open(tmp_path, "wb").close()
83         with _uploads_lock:
84             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86                 "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89     @router.put("/api/upload/{upload_id}/part/{index}")
90     async def upload_part(upload_id: str, index: int, request: Request):
91         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92         _, max_bytes, part_bytes, _ = _upload_cfg()
93         with _uploads_lock:
94             st = _uploads.get(upload_id)
95             if st is None:
96                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97             if index != st["parts"]:
98                 raise HTTPException(status_code=409,
99                                     detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101         received = 0
102         overflow = None
103         with open(st["tmp_path"], "ab") as fh:
104             async for chunk in request.stream():
105                 received += len(chunk)
106                 if received > part_bytes:
107                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                     break
109                 if st["bytes"] + received > max_bytes:

```

```

110         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111         break
112         fh.write(chunk)
113     if overflow:
114         _abort_upload(upload_id)
115         raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117     with _uploads_lock:
118         st["parts"] += 1
119         st["bytes"] += received
120     return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129             if st is None:
130                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131             if st["parts"] == 0 or st["parts"] != req.total_parts:
132                 raise HTTPException(status_code=409,
133                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134             final_path = os.path.join(upload_dir, st["filename"])
135             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136                 stem, ext = os.path.splitext(st["filename"])
137                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138             os.replace(st["tmp_path"], final_path)
139             with _uploads_lock:
140                 _uploads.pop(upload_id, None)
141             video_id = compute_video_id(final_path)
142             return {"path": os.path.abspath(final_path), "video_id": video_id,
143                     "size_bytes": os.path.getsize(final_path),
144                     "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:
151             known = upload_id in _uploads
152             if not known:
153                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154             _abort_upload(upload_id)
155             return {"status": "aborted", "upload_id": upload_id}
156

```

14. user (2026-06-21T14:40:11.507Z)

```

110     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
111     const ordinalOf = useMemo(() => {
112         const m = new Map<number, number>();
113         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
114         return (id: number) => m.get(id) ?? 0;
115     }, [rallies]);
116
117     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the

```

whole call.

```
118     const onBuild = async () => {
119         if (build.status === "building") return;
120         const cid = newCorrelationId();
121         const filename = reelFilename(reelName);
122         const ids = [...sel.selected];
123         if (!videoId) {
124             log("info", "build.no_video", { cid, selected: ids.length });
125             setBuild({
126                 status: "error",
127                 error: t("build.noVideo"),
128             });
129             return;
130         }
131         setBuild({ status: "building" });
132         log("info", "build.start", {
133             cid,
134             video_id: videoId,
135             filename,
136             selected: ids.length,
137             total_sec: Math.round(sel.totalSec),
138             below_floor: warning?.atRisk.length ?? 0,
139         });
140         try {
141             const res = await compileReel(videoId, ids, filename, cid);
142             const url = highlightUrl(videoId, filename);
143             setBuild({ status: "done", url, filename });
144             log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
145         } catch (err) {
146             const status = err instanceof ApiError ? err.status : undefined;
147             const detail = err instanceof ApiError && err.body ? parseDetail(err.body) :
String(err);
148             setBuild({ status: "error", error: detail });
149             log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });
150         }
151     };
152
153     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
154     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
155     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
156     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
157         const cid = newCorrelationId();
158         log("info", "query.parse", { cid, source, ...parseTrace(q) });
159
160         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
161         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
162         if (q.unavailable.length > 0) {
163             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
164         }
165
166         if (!q.recognized) {
167             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
168             return; // empty / unparseable ? leave the user's curation untouched
169         }

```

Both confirmed exactly as the tickets describe. Now let me verify: the site server file (index.js ? T2/B3 reference `server.js` which may be wrong), the wrangler/deploy config, App.tsx imports line, IngestPanel/useIngestJob, and whether `deploy/` exists.

16. user (2026-06-21T14:40:16.876Z)

```
1      // Cloudflare Worker for khelsutra.guru ? serves the static site and handles the
2      // Request-Access form: POST /api/request-access -> validate -> insert into D1 (binding
DB).
3      // Pure helpers are exported so they can be unit-tested without a live Worker/D1.
4
5      import { d1Store } from "./store.js";
6      import { cfEmailNotify } from "./notify.js";
7
8      const SEGMENTS = ["academy", "individual"];
9      const GPU = ["yes", "unsure", "no", "prefer-cloud"];
10     const COURTS = ["1", "2-4", "5+"];
11     const STORAGE = ["gdrive", "onedrive", "dropbox", "bucket", "local", "unsure"];
12
13     function str(v, max) {
14         if (v == null) return null;
15         const s = String(v).trim();
16         return s ? s.slice(0, max) : null;
17     }
18
19     // Honeypot: humans never fill the hidden "company" field.
20     export function isBot(p) {
21         return !(p && typeof p.company === "string" && p.company.trim() !== "");
22     }
23
24     export function validate(p) {
25         if (!p || typeof p !== "object") return { ok: false, error: "bad payload" };
26         const email = String(p.email || "").trim();
27         if (!/^[\^@\s]+\@[\^@\s]+\.[\^@\s]+$/i.test(email)) return { ok: false, error: "valid email
required" };
28         return { ok: true };
29     }
30
31     export function toRow(p, meta = {}) {
32         const storage = Array.isArray(p.storage)
33             ? p.storage.filter((s) => STORAGE.includes(s)).join(",")
34             : "";
35         return {
36             segment: SEGMENTS.includes(p.segment) ? p.segment : null,
37             name: str(p.name, 200),
38             email: str(p.email, 320),
39             city: str(p.city, 200),
40             org_name: str(p.org_name, 200),
41             courts: COURTS.includes(p.courts) ? p.courts : null,
42             has_gpu: GPU.includes(p.has_gpu) ? p.has_gpu : null,
43             storage,
44             message: str(p.message, 4000),
45             consent: p.consent ? 1 : 0,
46             country: str(meta.country, 8),
47             user_agent: str(meta.userAgent, 400),
48         };
49     }
50
51     function json(obj, status = 200) {
52         return new Response(JSON.stringify(obj), { status, headers: { "content-type":
"application/json" } });
53     }
54
55     export async function handleRequest(request, store, notify) {
56         let body;
```

```

57     try {
58         body = await request.json();
59     } catch {
60         return json({ ok: false, error: "invalid json" }, 400);
61     }
62     if (isBot(body)) return json({ ok: true }); // silently drop bots
63     const v = validate(body);
64     if (!v.ok) return json({ ok: false, error: v.error }, 400);
65
66     const r = toRow(body, {
67         country: request.headers.get("cf-ipcountry"),
68         userAgent: request.headers.get("user-agent"),
69     });
70
71     await store.insertAccessRequest(r);
72     try { await notify?.notify(r); } catch { /* email is best-effort; never fail the
request */ }
73
74     return json({ ok: true });
75 }
76
77 export default {
78     async fetch(request, env) {
79         const url = new URL(request.url);
80         if (url.pathname === "/api/request-access") {
81             if (request.method !== "POST") return json({ ok: false, error: "method not
allowed" }, 405);
82             return handleRequest(request, d1Store(env.DB), cfEmailNotify(env.NOTIFY,
env.NOTIFY_FROM, env.NOTIFY_TO));
83         }
84         return env.ASSETS.fetch(request); // everything else: static assets
85     },
86 };
87

```

17. user (2026-06-21T14:40:17.074Z)

```

1     // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2     // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3     //
4     // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5     // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6     // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7     // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8     // cancellable on unmount / user-cancel / a superseding submit.
9     import { useCallback, useEffect, useRef, useState } from "react";
10    import { getJob, processVideo } from "../api/client";
11    import { classifyError, type IngestError } from "../lib/errors";
12    import { log, newCorrelationId } from "../lib/log";
13    import type { JobStatus } from "../api/types";
14
15    const POLL_MIN_MS = 1000;
16    const POLL_MAX_MS = 5000;
17    const POLL_BACKOFF = 1.5;
18    const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20    export type IngestPhase =
21        | { status: "idle" }
22        | { status: "submitting" }

```

```

23     | { status: "polling"; progress: string | null }
24     | { status: "done"; videoId: string }
25     | { status: "error"; error: IngestError };
26
27     interface Run {
28         cancelled: boolean;
29         timer?: ReturnType<typeof setTimeout>;
30     }
31
32     export function useIngestJob(onLoaded?: (videoId: string) => void) {
33         const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34         const runRef = useRef<Run | null>(null);
35         // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36         const onLoadedRef = useRef(onLoaded);
37         onLoadedRef.current = onLoaded;
38
39         const stop = useCallback(() => {
40             const run = runRef.current;
41             if (run) {
42                 run.cancelled = true;
43                 if (run.timer) clearTimeout(run.timer);
44             }
45             runRef.current = null;
46         }, []);
47
48         // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49         useEffect(() => stop, [stop]);
50
51         const submit = useCallback(
52             (rawUri: string) => {
53                 const uri = rawUri.trim();
54                 if (!uri) {
55                     setPhase({ status: "error", error: { code: "invalidUri" } });
56                     return;
57                 }
58
59                 stop(); // supersede any prior run
60                 const run: Run = { cancelled: false };
61                 runRef.current = run;
62
63                 const cid = newCorrelationId();
64                 const startedAt = Date.now();
65                 setPhase({ status: "submitting" });
66                 log("info", "ingest.submit_start", { cid, uri });
67
68                 const fail = (error: IngestError, event: string) => {
69                     if (run.cancelled) return;
70                     setPhase({ status: "error", error });
71                     const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72                     log(level, event, { cid, uri, code: error.code, detail: error.detail });
73                 };
74
75                 const poll = (jobId: string, delay: number) => {
76                     run.timer = setTimeout(() => {
77                         void (async () => {
78                             if (run.cancelled) return;
79                             let job: JobStatus;
80                             try {
81                                 job = await getJob(jobId, cid);
82                             } catch (err) {
83                                 fail(classifyError(err), "ingest.poll_error");
84                             }
85                             return;

```

```

85         }
86         if (run.cancelled) return;
87
88         if (job.status === "done") {
89             const videoId = job.video_id ?? job.result?.video_id ?? null;
90             // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91             // a success-shaped FAILURE ? never load it, never hang.
92             if (job.result?.status === "failed" || !videoId) {
93                 fail({ code: "emptyResult" }, "ingest.empty_result");
94                 return;
95             }
96             setPhase({ status: "done", videoId });
97             log("info", "ingest.done", { cid, uri, video_id: videoId });
98             onLoadedRef.current?.(videoId);
99             return;
100         }
101         if (job.status === "failed") {
102             fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103             return;
104         }
105         // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106         setPhase({ status: "polling", progress: job.progress ?? null });
107         if (Date.now() - startedAt > POLL_CEILING_MS) {
108             fail({ code: "timeout" }, "ingest.poll_timeout");
109             return;
110         }
111         poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112     }());
113     }, delay);
114 };
115
116 void (async () => {
117     let jobId: string;
118     try {
119         const resp = await processVideo(uri, cid);
120         jobId = resp.job_id;
121     } catch (err) {
122         fail(classifyError(err), "ingest.submit_error");
123         return;
124     }
125     if (run.cancelled) return;
126     log("info", "ingest.queued", { cid, uri, job_id: jobId });
127     setPhase({ status: "polling", progress: null });
128     poll(jobId, POLL_MIN_MS);
129 }());
130 },
131 [stop],
132 );
133
134 const cancel = useCallback(() => {
135     stop();
136     setPhase({ status: "idle" });
137     log("info", "ingest.cancel", {});
138 }, [stop]);
139
140 const reset = useCallback(() => {
141     stop();
142     setPhase({ status: "idle" });
143 }, [stop]);
144
145 const busy = phase.status === "submitting" || phase.status === "polling";
146 return { phase, submit, cancel, reset, busy };

```



```
147     }
148
```

18. assistant (2026-06-21T14:40:28.139Z)

Important finding: the site is a **Cloudflare Worker** (``site/src/index.js`` with ``env.ASSETS.fetch``), NOT a Node ``server.js`` reading files off disk. The B3 ticket's claims about ``server.js:49-67``, ``server.js:26-30 TYPES map``, ``wrangler.toml:13-15``, ``server.test.js`` HTTP boot, and a VM-host ``server.js`` static handler are **stale/inaccurate** relative to this checkout. Let me verify the wrangler config, the App.tsx import line, and check the deploy/scripts reality.

19. user (2026-06-21T14:40:31.241Z)

```
=== wrangler ===
```

Cloudflare Worker for khelsutra.guru ? static site + Request-Access form API.

```
#
```

One-time setup (owner, our Cloudflare account):

npx wrangler d1 create khelsutra-access # copy the printed database_id below

npx wrangler d1 execute khelsutra-access --remote --file=schema.sql

npx wrangler deploy # or connect this repo via Workers Builds (git)

```
#
```

```
name = "hidden-king-khelsutra" # the git-connected Worker that owns khelsutra.guru ? keep in
sync with Cloudflare
main = "src/index.js"
compatibility_date = "2025-06-01"
```

```
[assets]
directory = "./public"
binding = "ASSETS"
```

```
[[d1_databases]]
binding = "DB"
database_name = "khelsutra-access"
database_id = "cd02c6bf-4bd6-4a33-a333-ced86e36b511" # from `wrangler d1 create khelsutra-
access`
```

Email-on-registration: Email Routing send binding -> a VERIFIED destination address.

```
[[send_email]]
name = "NOTIFY"
destination_address = "avi.dullu@gmail.com"
```

```
[vars]
NOTIFY_FROM = "noreply@khelsutra.guru"
NOTIFY_TO = "avi.dullu@gmail.com"
```

```
=== site root files ===
```

```
README.md
access.db
make_og.py
node_modules
package-lock.json
package.json
public
schema.sql
scripts
server.js
src
wrangler.toml
=== App.tsx import line 13 ===
```

```

import { useEffect, useMemo, useState } from "react";
import { useTranslation, Trans } from "react-i18next";
import { useRallySelection } from "../hooks/useRallySelection";
import { RallyGrid } from "../components/RallyGrid";
import { QueryBar } from "../components/QueryBar";
import { SelectionFooter } from "../components/SelectionFooter";
import { LanguageSwitcher } from "../components/LanguageSwitcher";
import { CompileResult, type BuildState } from "../components/CompileResult";
import { IngestPanel } from "../components/IngestPanel";
import { runQuery, type ParsedQuery } from "../lib/query";
import { floorRisk } from "../lib/floor";
import { reelFilename } from "../lib/format";
import { parseDetail } from "../lib/errors";
import { log, newCorrelationId } from "../lib/log";
import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount } from
"./api/client";
import type { EngineConfig, RallySegment } from "../api/types";

/** One compact, structured description of how a query was understood (for the trace log). */
function parseTrace(q: ParsedQuery) {
  return {

```

20. user (2026-06-21T14:40:33.655Z)

```

=== server.js anywhere? ===
./site/server.js
=== server.test.js ===
// [VM/droplet host harness] Boots the REAL server.js (node:http + node:sqlite) and
// round-trips it over HTTP ? the same proof we run on the droplet, automated. Keeps the
// repo compatible with BOTH hosts: src/index.test.js covers the Cloudflare (Worker+D1)
// path; this covers the Node+SQLite path. Run: node --test src/ (or: npm run test:vm)
import { test, before, after } from "node:test";
import assert from "node:assert/strict";
import { spawn } from "node:child_process";
import { createServer } from "node:net";
import { mkdtempSync, rmSync } from "node:fs";
import { tmpdir } from "node:os";
import { join } from "node:path";
import { DatabaseSync } from "node:sqlite";

function freePort() {
  return new Promise((res) => {
    const s = createServer();
    s.listen(0, "127.0.0.1", () => { const p = s.address().port; s.close(() => res(p)); });
  });
}

async function waitUp(url, ms = 10000) {
  const end = Date.now() + ms;
  while (Date.now() < end) {
    try { await fetch(url); return; } catch { await new Promise((r) => setTimeout(r, 150)); }
  }
  throw new Error("server.js did not start listening in time");
}

let proc, base, dbPath, dir;

before(async () => {
  const port = await freePort();
  dir = mkdtempSync(join(tmpdir(), "ks-vm-"));
  dbPath = join(dir, "access.db");
  base = `http://127.0.0.1:${port}`;
  const serverPath = join(import.meta.dirname, "..", "server.js");
  proc = spawn(process.execPath, [serverPath], {
    env: { ...process.env, PORT: String(port), DB_PATH: dbPath },
    stdio: "ignore",

```

```

});
await waitUp(`${base}/`);
});

after(() => {
  proc?.kill();
  try { rmSync(dir, { recursive: true, force: true }); } catch {}
=== deploy-config.test.js ===
// Deploy-config guard ? the live Cloudflare deploy is git-connected (merge to `master`
// auto-deploys via Workers Builds). A silent drift in wrangler.toml would break the LIVE
// site with NO test noticing: PR #5 was exactly this ? the wrangler `name` didn't match
// the connected Worker, so builds missed khelsutra.guru. These assertions pin the deploy
// invariants. String-based on purpose (the site is dependency-free; no TOML parser dep).
import { test } from "node:test";
import assert from "node:assert/strict";
import { readFileSync, existsSync } from "node:fs";
import { join } from "node:path";

```

```

=== provision-vm.sh head ===

```

```

#!/usr/bin/env bash

```

First-class, PROVIDER-AGNOSTIC provisioning for the KhelSutra site (Node + SQLite)
Run from your LOCAL machine against a FRESH Ubuntu VM (DigitalOcean / GCE / any)
it pushes site/ via `git archive` (PRIVATE-REPO-SAFE ? no clone, token, or secret on t
VM), installs Node, installs + starts a systemd service, opens the firewall, and VERIFIE
it is actually serving. Idempotent (safe to re-run). Pairs with teardown-vm.sh so a
destroy ? resume-on-a-different-provider cycle is clean and reproducible.

```

#
SSH_TARGET=root@1.2.3.4 [SSH_KEY=~/.ssh/khelsutra_droplet] \
[REMOTE_DIR=/srv/khelsutra-site] [PORT=8787] [NODE_MAJOR=24] [RESTORE_DB=1]
site/scripts/provision-vm.sh

```

```

#
NODE_MAJOR must be >= 24 so `node:sqlite` is unflagged (server.js imports it directl

```

```

set -euo pipefail
: "${SSH_TARGET:?set SSH_TARGET=user@host}"
KEY="${SSH_KEY:-$HOME/.ssh/khelsutra_droplet}"
DIR="${REMOTE_DIR:-/srv/khelsutra-site}"
PORT="${PORT:-8787}"
NODE_MAJOR="${NODE_MAJOR:-24}"
SITE="$(cd "$(dirname "$0")/.." && pwd)" # the site/ dir (script lives in
site/scripts/)
REPO="$(cd "$SITE/.." && pwd)" # repo root (for git archive)
SSH0=(-i "$KEY" -o StrictHostKeyChecking=accept-new -o ConnectTimeout=10)

echo "? [1/5] waiting for SSH at $SSH_TARGET ..."
for _ in $(seq 1 60); do ssh "${SSH0[@]}" "$SSH_TARGET" true 2>/dev/null && break || sleep 5;
done
ssh "${SSH0[@]}" "$SSH_TARGET" true # final attempt ? fail loud if still down

```

```

echo "? [2/5] pushing site/ via git archive (private-repo-safe; no secret on the VM) ..."
ssh "${SSH0[@]}" "$SSH_TARGET" "rm -rf '$DIR/site' && mkdir -p '$DIR/site'"

```

core.autocrlf=false forces LF in the archive even when the operator runs on Windows
autocrlf=true otherwise overrides .gitattributes eol=lf, producing CRLF on the VM and
byte-for-byte parity drift vs Cloudflare (which serves the canonical LF blobs).

```

git -C "$REPO" -c core.autocrlf=false archive HEAD:site | ssh "${SSH0[@]}" "$SSH_TARGET" "tar -x
-C '$DIR/site'"

```

```

if [ -n "${RESTORE_DB:-}" ]; then
    echo "? restoring state DB $RESTORE_DB ? $SSH_TARGET:$DIR/access.db (clean resume) ..."
    scp "${SSH0[@]}" -q "$RESTORE_DB" "$SSH_TARGET:$DIR/access.db"
fi

echo "? [3/5] remote setup (Node $NODE_MAJOR + systemd unit + firewall) ..."

```

21. assistant (2026-06-21T14:40:38.045Z)

`server.js` DOES exist at `site/server.js` (root, not `site/public/`). So the site is dual-host: Cloudflare Worker (`src/index.js` + `wrangler.toml [assets]`) AND a Node `server.js` for the VM/droplet. The B3 ticket's line references are off but the dual-host architecture is real. Let me verify the `server.js` TYPES map / path-jail claims and the engine binding/CORS claims for T2, plus the cleanup_caches tool for T11.

22. user (2026-06-21T14:40:39.007Z)

```

1      // Always-on VM host for the KhelSutra site ? dependency-free (Node 22+ only).
2      // Serves public/ and POST /api/request-access into SQLite (node:sqlite), using the
3      // SAME handler logic as the Cloudflare Worker (src/index.js). This is the portability
4      // fallback "if Cloudflare acts out" ? see docs/PORTABILITY.md.
5      //
6      //   PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
7      //
8      import { createServer } from "node:http";
9      import { readFile } from "node:fs/promises";
10     import { join, extname, normalize } from "node:path";
11     import { DatabaseSync } from "node:sqlite";
12     import { handleRequest } from "./src/index.js";
13     import { sqliteStore } from "./src/store.js";
14     import { noopNotify } from "./src/notify.js";
15
16     const PORT = Number(process.env.PORT) || 8080;
17     const ROOT = import.meta.dirname;
18     const PUBLIC = join(ROOT, "public");
19     const DB_PATH = process.env.DB_PATH || join(ROOT, "access.db");
20
21     const db = new DatabaseSync(DB_PATH);
22     db.exec(await readFile(join(ROOT, "schema.sql"), "utf8")); // CREATE TABLE IF NOT EXISTS
? idempotent
23     const store = sqliteStore(db);
24     const notify = noopNotify(); // VM host: no email yet (add SMTP later ? see
docs/PORTABILITY.md)
25
26     const TYPES = {
27       ".html": "text/html; charset=utf-8", ".png": "image/png", ".svg": "image/svg+xml",
28       ".css": "text/css", ".js": "text/javascript", ".ico": "image/x-icon", ".txt":
"text/plain",
29       ".mp4": "video/mp4", ".webm": "video/webm", ".gif": "image/gif", ".jpg": "image/jpeg",
".jpeg": "image/jpeg",
30     };
31
32     async function toRequest(req) {
33       const url = `http://${req.headers.host || "localhost"}${req.url}`;
34       let body;
35       if (req.method !== "GET" && req.method !== "HEAD") {
36         const chunks = [];
37         for await (const c of req) chunks.push(c);
38         body = Buffer.concat(chunks);
39       }
40       return new Request(url, { method: req.method, headers: req.headers, body });
41     }
42
43     async function send(res, response) {
44       res.statusCode = response.status;

```

```

45     for (const [k, v] of response.headers) res.setHeader(k, v);
46     res.end(Buffer.from(await response.arrayBuffer()));
47 }
48
49 async function serveStatic(pathname, res) {
50     const rel = pathname === "/" ? "index.html"
51       : pathname === "/capture" ? "capture.html"
52       : pathname === "/own-model" ? "own-model.html"
53       : pathname === "/flyer" ? "flyer.html"
54       : pathname === "/flyer-kn" ? "flyer-kn.html"
55       : pathname === "/flyer-hi" ? "flyer-hi.html"
56       : pathname.replace(/^\/+/, "");
57     const file = normalize(join(PUBLIC, rel));
58     if (!file.startsWith(PUBLIC)) { res.statusCode = 403; return res.end("forbidden"); }
59     try {
60         const data = await readFile(file);
61         res.setHeader("content-type", TYPES[extname(file)] || "application/octet-stream");
62         res.end(data);
63     } catch {
64         res.statusCode = 404;
65         res.end("not found");
66     }
67 }
68
69 createServer(async (req, res) => {
70     try {
71         const pathname = decodeURIComponent(new URL(`http://x${req.url}`).pathname);
72         if (pathname === "/api/request-access") {
73             if (req.method !== "POST") { res.statusCode = 405; return
res.end('{"ok":false,"error":"method not allowed"}'); }
74             return await send(res, await handleRequest(await toRequest(req), store, notify));
75         }
76         return await serveStatic(pathname, res);
77     } catch (err) {
78         res.statusCode = 500;
79         res.end('{"ok":false,"error":"server error"}');
80         console.error(err);
81     }
82 }).listen(PORT, () => console.log(`khelsutra site listening on :${PORT} (sqlite:
${DB_PATH})`));
83

```

23. user (2026-06-21T14:40:41.963Z)

```

=== uvicorn bind ===
20:from fastapi.middleware.cors import CORSMiddleware
58:# CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by browsers
per the
63:    CORSMiddleware,
64:    allow_origins=cors_origins_from_env(),
756:        uvicorn.run(app, host="127.0.0.1", port=args.port)
=== cleanup_caches exists? ===
-rw-r--r-- 1 avidu 197609 11866 Jun 13 21:02 tools/cleanup_caches.py
=== retention_sweep exists? ===
no retention_sweep.py
=== gcs storage ===
__init__.py
__pycache__
base.py
gcs.py
gdrive.py
local.py
ref.py
s3.py

```

24. assistant (2026-06-21T14:40:47.614Z)

All confirmed. `server.js:26-30` TYPES map has NO `.json` entry (confirmed ? falls to octet-stream). `server.js:57-58` path-jails to PUBLIC (confirmed). Engine binds `127.0.0.1` at main.py:756 (confirmed). `tools/cleanup\_caches.py` exists; `retention\_sweep.py` is net-new (confirmed). Let me do final verification on the remaining specifics: the SecurityConfig allowed_video_root warning, the LOCAL_APPLIANCE doc, the SPA component anchor lines for A4/A8, and the cleanup_caches structure for T11.

25. user (2026-06-21T14:40:50.708Z)

```
=== models.py allowed_video_root / upload_dir ===
283:     allowed_video_root: Optional[str] = None
287:     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288:     # contain upload_dir or /api/process will reject the uploaded paths.
289:     upload_dir: str = "output/uploads"
293:     max_part_mb: int = 95
297:     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-
user
303:     edge_auth_enabled: bool = False
=== cleanup_caches helpers ===
9:DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass `--apply` to delete.
12:     python -m tools.cleanup_caches --apply                # delete purge candidates
13:     python -m tools.cleanup_caches --keep-video 89605e80ed01 # protect one video's cache
14:     python -m tools.cleanup_caches --older-than 7          # only purge entries >7 days old
24:import argparse
40:def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
72:class Entry:
81:     def category(self) -> Tuple[str, bool, bool]:
82:         return classify(self.name, self.is_dir, self.location)
85:def plan_deletions(entries: List[Entry], *, keep_video: Optional[str], older_than_days:
float,
104:def human(n: int) -> str:
113:def _wsl_lines(distro: str, cmd: str) -> List[str]:
123:def scan_wsl(distro: str, stage_dir: str) -> List[Entry]:
160:def _dir_size(path: str) -> int:
171:def scan_windows(output_dir: str) -> List[Entry]:
192:def _delete(e: Entry, distro: str) -> bool:
206:def _q(path: str) -> str:
212:def _report(title: str, rows: List[Entry]) -> int:
227:def main(argv: Optional[List[str]] = None) -> int:
228:     ap = argparse.ArgumentParser(description="Report/clean regenerable pipeline caches (dry-
run by default).")
229:     ap.add_argument("--apply", action="store_true", help="actually delete (default: report
only)")
230:     ap.add_argument("--keep-video", default=None, help="protect entries whose name contains
this id/substring")
231:     ap.add_argument("--older-than", type=float, default=0.0, help="only purge entries older
than N days")
244:     entries = scan_wsl(args.distro, args.stage_dir) + scan_windows(args.output_dir)
245:     to_delete, kept = plan_deletions(
253:         f"? run `python -m tools.cleanup_caches` to review, `--apply` to delete.")
261:     print("\nDRY RUN ? nothing deleted. Re-run with --apply to delete the PURGE list.")
```

26. user (2026-06-21T14:40:53.343Z)

```
Exit code 2
=== LOCAL_APPLIANCE doc ===
-rw-r--r-- 1 avidu 197609 29153 Jun 21 19:57 docs/LOCAL_APPLIANCE_ARCHITECTURE.md
=== FRIENDLY doc ===
docs/FRIENDLY_AND_MULTILINGUAL.md
```

27. assistant (2026-06-21T14:40:58.599Z)

All anchors confirmed. CompileResult line 21 confirmed (``build.status === "idle" || build.status === "building" return null``). RallyGrid shots chip uses ``title={t("grid.shotsTitle")}`` with visible ``grid.shots`` text. The en catalog has ``result``, ``ingest``, ``selection``, ``footer``, ``grid`` blocks but NO ``errors`` block yet (A5 net-new).

I have verified all load-bearing claims. Let me check the cross-cutting App.tsx bulk buttons and confirm whether errors block exists, then finalize.

30. user (2026-06-21T14:41:10.172Z)

```
=== errors block in en? ===
0
=== ingest error keys present (A5 migrates these) ===
60:   "failed": "Build failed",
79:   "failed": "Couldn't process the video: {error}",
80:   "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine
running KhelSutra.",
81:   "networkError": "Can't reach KhelSutra. Is it running?",
82:   "emptyResult": "Processing finished but found no rallies in this video. Try a clearer
match recording.",
83:   "timeout": "This is taking longer than expected. The job may still be running ? reload
with the video link, or try again."
=== App.tsx bulk buttons ~247 ===
</div>

<div className="mt-6 flex flex-wrap gap-2 text-sm">
  <button type="button" onClick={() => onBulk("select_all")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">{t("bulk.selectAll")}</button>
  <button type="button" onClick={() => onBulk("clear")} className="rounded-full border
border-black/15 px-3 py-1 hover:border-green">{t("bulk.clear")}</button>
  <button type="button" onClick={() => onBulk("invert")} className="rounded-full border
border-black/15 px-3 py-1 hover:border-green">{t("bulk.invert")}</button>
</div>

<div className="mt-5">
```

31. assistant (2026-06-21T14:41:22.979Z)

All claims verified. I now have a complete, accurate picture. Key findings on accuracy of the tickets:

- ****All core technical claims hold****: engine ``127.0.0.1`` bind (main.py:756), CORS default ``*``, ``request()`` content-type trap (client.ts:42), upload endpoints exactly as described, ``App.tsx`` onBuild uses raw ``parseDetail`/`String(err)`` (lines 145-150), CompileResult early-return at line 21, RallyGrid shots ``title=`` tooltip, no ``errors.*`` block yet, ingest error keys present to migrate, ``cleanup_caches.py`` exists with the described scaffolding, ``retention_sweep.py`` is net-new, SecurityConfig ``allowed_video_root`/`upload_dir`` warning at models.py:287-289, ``max_part_mb=95``, ``edge_auth_enabled=False``.
- ****B3's host model is more nuanced than the ticket's line refs****: the site is ****dual-host**** ? a Cloudflare Worker (``site/src/index.js`` + ``wrangler.toml [assets]``) AND a Node ``server.js`` (at ``site/server.js``, not ``public/``). The ticket's specific line numbers (server.js:49-67/26-30, wrangler.toml:13-15) are slightly off but the substance is correct: ``server.js`` TYPES map has no ``json``, path-jails to PUBLIC at :57-58, and the dual-host parity discipline is real (``deploy-config.test.js``, ``server.test.js``). The B3 work is valid; the file refs just need re-confirming during implementation.
- ****`deploy/`** does not exist yet** in khelsutra ? T2's ``deploy/appliance/`` are all genuinely net-new.
- ****Cross-repo split confirmed****: A1-A8/B3/T4/T2 live in khelsutra; T11 lives in the BHI engine repo (``tools/``). LOCAL_APPLIANCE_ARCHITECTURE.md lives in khelsutra; I18N_PLAN.md in BHI engine.

32. user (2026-06-21T14:42:43.959Z)

Structured output provided successfully

33. assistant (2026-06-21T14:42:54.307Z)

I've completed the analysis and returned the structured handoff plan. Summary of what I verified and produced:

****Verification (all load-bearing claims checked against both checkouts):****

- Engine `127.0.0.1:8000` bind (main.py:756), CORS default `\*`, chunked-upload flow in `backend/api/uploads.py`, `request()` content-type trap (client.ts:42), `App.tsx` onBuild raw error path (145-150), CompileResult line-21 early return, RallyGrid `title=` tooltip, no `errors.\*` block yet but `ingest.\*` error keys present to migrate, `cleanup\_caches.py` scaffold + net-new `retention\_sweep.py`, SecurityConfig `allowed\_video\_root` ? upload_dir` warning (models.py:287-289), `max_part_mb=95`.

****Two corrections to the tickets surfaced in the output:****

1. B3's file refs are stale ? the site is ****dual-host**** (Cloudflare Worker `site/src/index.js` + Node `site/server.js` at root, not `public/`); the substance (no `.json` in TYPES map, PUBLIC path-jail, mandatory catalog copy/hoist) holds, but line numbers need re-confirming during implementation.
2. `deploy/` doesn't exist yet ? T2's `deploy/appliance/\*` are genuinely net-new.

The structured result delivers: execution order (A5?A4?A8?B3 for the SPA UX block, then T4?T2?T11 for the North Star), cross-item coordination (the 3 catalogs + App.tsx + IngestPanel are the serialization hotspots; the T4?T2 `allowed\_video\_root` handshake; the two-repo split), the North-Star summary (what exists vs net-new + cheapest path), and risks (T11 as a launch blocker before the gate opens, the GEMINI_API_KEY rotation, the three-condition box-safety AND, the `request()` part-PUT trap).